

ĐẠI HỌC QUỐC GIA TP.HCM  
TRƯỜNG ĐẠI HỌC BÁCH KHOA  
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH  
\_\_\_\_\_ \*



**LUẬN VĂN TỐT NGHIỆP ĐẠI HỌC**

**PHÁT HIỆN POWERSHELL SCRIPT  
TẤN CÔNG THEO DẠNG FILELESS  
MALWARE Ở PHÍA NGƯỜI DÙNG  
BẰNG PHƯƠNG PHÁP SANDBOX**

Ngành: KHOA HỌC MÁY TÍNH

HỘI ĐỒNG: KHOA HỌC MÁY TÍNH 21

GVHD: TS. Nguyễn An Khương

KS. Nguyễn Anh Khoa – VNG Corp

KS. Võ Văn Tiến Dũng – LINE Corp

GVPB: TS. Trương Tuấn Anh

o0o

SVTH: Đỗ Đình Phú Quý – 2014289

TP. HỒ CHÍ MINH, THÁNG 06/2024

KHOA: KH & KT Máy tính  
BỘ MÔN: KHMT

**NHIỆM VỤ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP**  
*Chú ý: Sinh viên phải dán tờ này vào trang nhất của bản thuyết trình*

HỌ VÀ TÊN: Đỗ Đình Phú Quý  
NGÀNH: KHMT

MSSV: 2014289  
LỚP: MT20KH10

**1. Đầu đề luận văn/ đồ án tốt nghiệp: “DETECTING FILELESS- BASED POWERSHELL MALWARE AT END-USER WITH SANDBOX”**

**2. Nhiệm vụ (yêu cầu về nội dung và số liệu ban đầu):**

- Nghiên cứu về mã độc loại Fileless PowerShell và một số kỹ thuật được sử dụng trong loại mã độc này.
- Tìm hiểu về hành vi, dấu hiệu của một số loại mã độc phân tán khác.
- Nghiên cứu về kỹ thuật Hooking, Sandbox cho quá trình hiện thực công cụ.
- Xây dựng công cụ cô lập quá trình hoạt động của mã độc, ứng dụng trong việc phát hiện hành vi độc hại của mã độc Ransomware dạng PowerShell.
- Viết báo cáo chi tiết về quá trình hiện thực luận văn.

**3. Ngày giao nhiệm vụ: 08/01/2024**

**4. Ngày hoàn thành nhiệm vụ: 20/05/2024**

**5. Họ tên giảng viên hướng dẫn:**

- 1) Nguyễn An Khương
- 2) Nguyễn Anh Khoa
- 3) Võ Văn Tiến Dũng

**Phản hướng dẫn:**

Hướng dẫn chung và giám sát thực hiện  
Hướng dẫn kỹ thuật  
Hướng dẫn kỹ thuật

Nội dung và yêu cầu LVTN/ ĐATN đã được thông qua Bộ môn.

Ngày ..... tháng ..... năm .....  
**CHỦ NHIỆM BỘ MÔN**  
(Ký và ghi rõ họ tên)

**GIẢNG VIÊN HƯỚNG DẪN CHÍNH**  
(Ký và ghi rõ họ tên)

Nguyễn An Khương

*PHẦN DÀNH CHO KHOA, BỘ MÔN:*

Người duyệt (chấm sơ bộ):

Đơn vị:

Ngày bảo vệ:

Điểm tổng kết:

Nơi lưu trữ LVTN/ĐATN:

Ngày 17 tháng 5 năm 2024

**PHIẾU ĐÁNH GIÁ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP**  
(Dành cho người hướng dẫn)

1. Họ và tên SV: Đỗ Đình Phú Quý

MSSV: 2014289

Ngành (chuyên ngành): Khoa học Máy tính

2. Đề tài: “Phát hiện Powershell Script tấn công theo dạng Fileless Malware ở phía người dùng bằng phương pháp Sandbox”

3. Họ tên người hướng dẫn: Nguyễn An Khương, Nguyễn Anh Khoa và Võ Văn Tiến Dũng

4. Tổng quát về bản thuyết minh:

Số trang: 66 trang

Số chương: 8 chương và 1 phụ lục

Số bảng số liệu: 5 bảng

Số hình vẽ: 21 hình vẽ

Số tài liệu tham khảo: 22 tài liệu

Phần mềm tính toán: Không

Hiện vật (sản phẩm): Không

5. Những ưu điểm chính của LV/ ĐATN:

- Sinh viên đã tìm hiểu một số cách thức phát tán mã độc phổ biến trong các cuộc tấn công Ransomware, và tìm được điểm hạn chế ở các kỹ thuật hiện tại trong việc phát hiện mã độc không dùng file (Fileless Malware) điển hình với PowerShell.
- Sinh viên đã đề xuất kỹ thuật và thực hiện để khắc phục được các hạn chế trong các kỹ thuật hiện tại.
- Sinh viên đã tìm hiểu và biết các cách thiết kế một môi trường cô lập (sandbox).
- Sinh viên đã vận dụng khả năng cô lập của sandbox để thiết kế hệ thống cô lập mã thực thi nhằm thu thập hành vi của mã thực thi.
- Sinh viên rất giỏi kỹ thuật và biết cách sử dụng thành thạo các công cụ lập trình và các công cụ liên quan để thực hiện đề tài.

6. Những thiếu sót chính của LV/ĐATN:

- Môi trường cô lập (sandbox) của sinh viên cần phát triển thêm, vì công cụ vẫn chưa hoàn toàn cô lập môi trường.
- Sinh viên chưa thực hiện các đánh giá trên các mẫu (mã độc) thực tế, ảnh hưởng đến đánh giá của đề tài.
- Công cụ còn nhiều hạn chế, điển hình yêu cầu người sử dụng thao tác tay.
- Thang đánh giá mã độc chưa rõ ràng, hệ thống đánh giá mã độc chưa thể hiện được lý do tại sao file/script được xem là mã độc, chẳng hạn, đếm số lượng các file truy cập nếu lớn hơn 5 thì cho là “malicious”.

7. Đề nghị: Được bảo vệ ☒

Bổ sung thêm để bảo vệ ☐

Không được bảo vệ ☐

8. Các câu hỏi SV phải trả lời trước Hội đồng:

- a. Tại sao không thiết kế một hệ thống Sandbox hoàn toàn mà dựa vào “hook” các lời gọi hàm?
- b. Có thể sử dụng cho Fileless Malware khác ngoài PowerShell, như Javascript?
- c. Triển khai thực tế sẽ như thế nào? Việc tích hợp vào các hệ thống đã có sẵn như thế nào?

9. Đánh giá chung (bằng chữ: Xuất sắc, Giỏi, Khá, TB): Giỏi

Điểm : 9.3/10

Ký tên (ghi rõ họ tên)

Nguyễn An Khương

Ngày 31 tháng 05 năm 2024

**PHIẾU ĐÁNH GIÁ LUẬN VĂN/ ĐỒ ÁN TỐT NGHIỆP**

(Dành cho người hướng dẫn/phản biện)

1. Họ và tên SV: **Đỗ Đình Phú Quý**  
MSSV: 2014289  
Ngành (chuyên ngành): Khoa học Máy tính
2. Đề tài: PHÁT HIỆN POWERSHELL SCRIPT TẤN CÔNG THEO DẠNG FILELESS MALWARE Ở PHÍA NGƯỜI DÙNG BẰNG PHƯƠNG PHÁP SANDBOX
3. Họ tên người hướng dẫn/phản biện: TS. Trương Tuấn Anh
4. Tổng quát về bản thuyết minh:
- |                        |                     |
|------------------------|---------------------|
| Số trang:              | Số chương:          |
| Số bảng số liệu        | Số hình vẽ:         |
| Số tài liệu tham khảo: | Phần mềm tính toán: |
| Hiện vật (sản phẩm)    |                     |

5. Những ưu điểm chính của LVTN:

Đề tài tìm hiểu và hiện thực một công cụ có khả năng phát hiện được mã độc Powershell Ransomware và hành vi của một số mã độc phân tán khác. Thông qua việc giám sát, kiểm tra, thay đổi được các Input và Output của hàm API bằng kỹ thuật Hooking, công cụ có thể biết được hành vi của một chương trình. Từ đó có thể đưa ra một kết luận về hành vi của chương trình PowerShell cần được kiểm tra có phải là Ransomware hoặc là một hành vi độc hại hay không. Tác giả đã có tìm hiểu các công trình liên quan và các kỹ thuật, công nghệ cần thiết để đề xuất, thiết kế và xây dựng hệ thống.

6. Những thiếu sót chính của LVTN:

Tác giả cần lập luận rõ lợi ích của việc xây dựng hệ thống của tác giả thay vì sử dụng các cơ chế phân tích mã độc hiện tại. Thêm nữa, các tiêu chí để quyết định các hành động là của mã độc hay không cần phải được nghiên cứu và lập luận rõ.

7. Đề nghị: Được bảo vệ ☒ Bỏ sung thêm để bảo vệ ☐ Không được bảo vệ ☐

8. 3 câu hỏi SV phải trả lời trước Hội đồng:

9. Đánh giá chung (bằng chữ: giỏi, khá, TB): Giỏi

Điểm: 7.5 /10

Ký tên (ghi rõ họ tên)

**TS. Trương Tuấn Anh**

## Lời cam đoan

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi dưới sự hướng dẫn trực tiếp của TS. Nguyễn An Khương, KS. Nguyễn Anh Khoa và KS. Võ Văn Tiến Dũng. Nội dung nghiên cứu và các kết quả đều là trung thực và chưa từng được công bố trước đây. Các số liệu được sử dụng cho quá trình phân tích, nhận xét được chính tôi thu thập từ nhiều nguồn khác nhau và sẽ được ghi rõ trong phần tài liệu tham khảo. Ngoài ra, tôi cũng có sử dụng một số nhận xét, đánh giá và số liệu của các tác giả, cơ quan tổ chức khác. Tất cả đều có trích dẫn và chú thích nguồn gốc. Nếu phát hiện có bất kì sự gian lận nào, tôi xin hoàn toàn chịu trách nhiệm về nội dung luận văn tốt nghiệp của mình. Trường Đại học Bách Khoa - Đại học Quốc gia TP. HCM không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện.

## Lời cảm ơn / Lời ngỏ

Đầu tiên, tôi xin bày tỏ lòng biết ơn và lời cảm ơn sâu sắc nhất đến những người hướng dẫn trực tiếp của tôi là thầy Nguyễn An Khương, anh Nguyễn Anh Khoa và anh Võ Văn Tiến Dũng, những người đã giúp tôi thấy rõ hơn về hướng đi của mình. Mọi người là những người truyền cảm hứng, động viên, tạo cơ hội giúp tôi tiến bộ hơn trên con đường học tập và sự nghiệp bằng sự kiên nhẫn, nhiệt tình và kiến thức chuyên môn sâu rộng. Chính sự dìu dắt, hướng dẫn tận tình đó đã giúp tôi có nền tảng vững vàng hơn về kiến thức và có niềm tin hơn vào bản thân mình, là cơ sở để tôi có thể chinh phục nhiều mục tiêu khác trong cuộc sống. Bên cạnh đó, tôi cũng muốn gửi một lời cảm ơn chân thành đến các thầy cô trường Đại học Bách Khoa - Đại học Quốc gia TP. HCM nói chung và các thầy cô trong khoa Khoa học & Kỹ thuật Máy tính nói riêng, những người đã truyền đạt cho tôi nhiều kiến thức quý giá trong chặng đường bốn năm đại học vừa qua. Bách Khoa đã tạo cho tôi một môi trường vô cùng tốt để học tập và phát triển.

Tiếp theo, tôi xin gửi lời cảm ơn sâu sắc đến cha và mẹ tôi. Những người đã luôn động viên, tin tưởng và hỗ trợ cho tôi rất nhiều để tôi có thể tập trung nghiên cứu và hoàn thiện luận văn này.

Tôi cũng xin gửi lời cảm ơn đến tất cả những thành viên trong Câu lạc bộ An toàn thông tin Bách Khoa (BKISC), những người bạn, người anh và người em. Tại đây, tôi không chỉ học thêm nhiều về kiến thức chuyên môn, mà còn được duy trì sở thích của mình, được tham gia nhiều cuộc thi để nâng cao kiến thức, xây dựng cho mình thêm nhiều kỹ năng và học cách giải quyết các vấn đề khác trong cuộc sống.

Ngoài ra, tôi cũng muốn gửi một lời cảm ơn chân thành đến công ty VNG Corporation nơi tôi đang thực tập. Tại đây tôi được học tập và làm việc cùng những con người tài giỏi, một môi trường làm việc năng động và tuyệt vời. Mọi người tại đây cũng đã hỗ trợ tôi rất nhiệt tình trong suốt quá trình làm luận văn.

# Tóm tắt đồ án

Trong thời đại kỹ thuật số ngày nay, mã độc Ransomware đang không ngừng gia tăng cả về số lượng và mức độ phức tạp. Không chỉ xuất hiện với rất nhiều biến thể mới, Ransomware còn áp dụng hàng loạt kỹ thuật tiên tiến để vượt qua các giải pháp bảo mật truyền thống. Một trong những phương thức triển khai Ransomware hiện đại và đáng lo ngại nhất là thông qua PowerShell, một công cụ mạnh mẽ của Windows.

Nguyên nhân cho việc mã độc thường được triển khai bằng PowerShell là do hệ thống bảo mật thường dựa vào các signature trong các mẫu mã độc cũ để phát hiện mã độc. Nhưng với PowerShell, kẻ tấn công có thể dễ dàng vượt qua các biện pháp này bằng cách sử dụng kỹ thuật làm rối mã (Obfuscation). Kỹ thuật Obfuscation làm cho mã PowerShell trở nên khó đọc và khó hiểu, giấu đi các dấu hiệu nhận diện quen thuộc, khiến cho việc phát hiện bằng các biện pháp truyền thống trở nên vô cùng khó khăn. Do đó cần phải thực hiện phân tích động, tức là cho mã độc thực thi và quan sát được các hành vi độc hại đó. Có rất nhiều cách để có thể thực hiện, tuy nhiên trong luận văn này chúng tôi sẽ đưa ra hai cách tiếp cận sau: giả lập chương trình (Emulating) và thực thi mã độc trong một trường ảo (Sandbox). Với cách tiếp cận bằng Emulating, đòi hỏi cần rất nhiều kiến thức về ngôn ngữ, hệ thống máy tính và hạn chế về mặt thời gian. Do đó chúng tôi đã chọn cách tiếp cận thứ hai.

Với kỹ thuật thực thi mã độc trên Sandbox, thì chúng tôi đối diện với hai bài toán nhỏ hơn. Thứ nhất, chúng tôi có thể xây dựng một môi trường hoàn toàn cô lập, tương tự như một máy ảo. Cách tiếp cận thứ hai là chặn một số chức năng của mã độc có thể tác động tới hệ thống thật. Với cách tiếp cận đầu tiên, việc tạo ra một môi trường cô lập hoàn toàn như một máy ảo yêu cầu sự đầu tư lớn về thời gian và kiến thức chuyên sâu về hệ thống máy tính. Quá trình này đòi hỏi phải thiết lập và cấu hình các yếu tố ảo hóa, đảm bảo rằng mọi hành vi của mã độc đều được giới hạn trong môi trường an toàn, không thể gây hại cho hệ thống thật. Mặt khác, cách tiếp cận thứ hai tập trung vào việc ngăn chặn các chức năng nguy hiểm của mã độc. Bằng cách ngăn chặn và giám sát các hoạt động mà mã độc có thể sử dụng để tấn công vào hệ thống, từ đó chúng ta có thể giảm thiểu rủi ro mà không cần phải xây dựng một môi trường hoàn toàn cô lập. Sau khi tìm hiểu, chúng tôi đã thực hiện và đã đạt được một số kết quả khả quan.

# Mục lục

|                                                                  |          |
|------------------------------------------------------------------|----------|
| Danh sách bảng                                                   | x        |
| Danh sách hình vẽ                                                | xi       |
| Danh mục từ viết tắt                                             | xii      |
| <b>1 Giới thiệu</b>                                              | <b>1</b> |
| 1.1 Giới thiệu đề tài . . . . .                                  | 1        |
| 1.2 Mục tiêu và phạm vi của đề tài . . . . .                     | 2        |
| 1.3 Cấu trúc luận văn . . . . .                                  | 3        |
| <b>2 Kiến thức nền tảng</b>                                      | <b>5</b> |
| 2.1 Mã độc . . . . .                                             | 5        |
| 2.1.1 Một số loại mã độc . . . . .                               | 5        |
| 2.1.2 Ransomware . . . . .                                       | 6        |
| 2.1.3 Fileless Malware . . . . .                                 | 7        |
| 2.2 PowerShell . . . . .                                         | 8        |
| 2.2.1 PowerShell Obfuscation . . . . .                           | 9        |
| 2.2.2 Ransomware sử dụng Fileless PowerShell làm Dropper Malware | 12       |
| 2.3 Các kỹ thuật phân tích mã độc . . . . .                      | 12       |
| 2.3.1 Phân tích tĩnh . . . . .                                   | 12       |
| 2.3.2 Phân tích động . . . . .                                   | 13       |
| 2.4 Định dạng file Portable Executable . . . . .                 | 13       |
| 2.4.1 Cấu trúc cơ bản . . . . .                                  | 14       |
| 2.4.2 DOS Header . . . . .                                       | 14       |
| 2.4.3 DOS Stub . . . . .                                         | 15       |
| 2.4.4 NT Headers . . . . .                                       | 15       |
| 2.4.5 Section Table . . . . .                                    | 18       |
| 2.5 Kỹ thuật chèn mã (Hooking) . . . . .                         | 19       |
| 2.5.1 IAT Hooking . . . . .                                      | 20       |
| 2.5.2 Inline Hooking . . . . .                                   | 21       |



|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| 2.6      | Môi trường ảo hóa (Sandbox) . . . . .                  | 22        |
| <b>3</b> | <b>Các công cụ và nghiên cứu liên quan</b>             | <b>24</b> |
| 3.1      | Các công cụ . . . . .                                  | 24        |
| 3.1.1    | Frida . . . . .                                        | 24        |
| 3.1.2    | Yara . . . . .                                         | 25        |
| 3.2      | Một số nghiên cứu liên quan . . . . .                  | 26        |
| 3.2.1    | Nhận dạng Fileless PowerShell Malware . . . . .        | 26        |
| 3.2.1.1  | Microsoft Copilot . . . . .                            | 27        |
| 3.2.1.2  | Antimalware Scan Interface . . . . .                   | 27        |
| 3.2.1.3  | AnyRun . . . . .                                       | 28        |
| 3.2.2    | Kỹ thuật chống máy ảo (Anti Virtual Machine) . . . . . | 28        |
| <b>4</b> | <b>Đề xuất giải pháp</b>                               | <b>30</b> |
| 4.1      | Đề xuất phương pháp . . . . .                          | 30        |
| 4.2      | Ngôn ngữ và các thư viện được sử dụng . . . . .        | 31        |
| <b>5</b> | <b>Phân tích các hàm API</b>                           | <b>33</b> |
| 5.1      | Files . . . . .                                        | 33        |
| 5.1.1    | CreateFile . . . . .                                   | 33        |
| 5.1.2    | WriteFile và ReadWrite . . . . .                       | 34        |
| 5.1.3    | DeleteFile . . . . .                                   | 34        |
| 5.1.4    | MoveFile và CopyFile . . . . .                         | 34        |
| 5.2      | Registry . . . . .                                     | 35        |
| 5.2.1    | RegCreateKey và RegCreateKeyEx . . . . .               | 35        |
| 5.2.2    | RegOpenKey và RegOpenKeyEx . . . . .                   | 35        |
| 5.2.3    | RegSetValue và RegSetValueEx . . . . .                 | 36        |
| 5.2.4    | RegDeleteKey và RegDeleteKeyEx . . . . .               | 36        |
| 5.2.5    | RegDeleteValue và RegDeleteKeyValue . . . . .          | 37        |
| 5.3      | Process . . . . .                                      | 37        |
| 5.3.1    | CreateProcess . . . . .                                | 37        |
| 5.3.2    | OpenProcess . . . . .                                  | 38        |
| 5.4      | Internet . . . . .                                     | 38        |
| 5.4.1    | GetAddrInfo . . . . .                                  | 38        |
| 5.4.2    | WinHttpGetProxyForUrl . . . . .                        | 38        |
| <b>6</b> | <b>Hiện thực công cụ</b>                               | <b>39</b> |
| 6.1      | Phân tích hệ thống . . . . .                           | 39        |
| 6.1.1    | Sandbox . . . . .                                      | 39        |
| 6.1.1.1  | ObjectsManager . . . . .                               | 40        |
| 6.1.1.2  | YaraScanner . . . . .                                  | 41        |
| 6.1.2    | Interceptor . . . . .                                  | 42        |
| 6.1.3    | Runner . . . . .                                       | 43        |
| 6.1.4    | Report . . . . .                                       | 44        |
| 6.1.5    | Detector . . . . .                                     | 45        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 6.2      | Quá trình xử lý dữ liệu của các thành phần trong hệ thống . . . . . | 47        |
| 6.2.1    | Quá trình xử lý CreateFile và OpenFile . . . . .                    | 47        |
| 6.2.2    | Quá trình xử lý WriteFile . . . . .                                 | 48        |
| 6.2.3    | Quá trình xử lý ReadFile . . . . .                                  | 49        |
| 6.2.4    | Quá trình xử lý DeleteFile, CopyFile và MoveFile . . . . .          | 50        |
| 6.2.5    | Các hàm API liên quan tới Registry . . . . .                        | 50        |
| 6.2.6    | Các hàm API liên quan tới Process và Internet . . . . .             | 51        |
| <b>7</b> | <b>Kiểm thử và Đánh giá công cụ</b>                                 | <b>53</b> |
| 7.1      | Kiểm thử . . . . .                                                  | 53        |
| 7.2      | So sánh giữa các công cụ đã được nghiên cứu . . . . .               | 56        |
| 7.3      | Đánh giá . . . . .                                                  | 57        |
| 7.3.1    | Một số tiêu chí đánh giá . . . . .                                  | 57        |
| 7.3.2    | Kết quả đạt được . . . . .                                          | 57        |
| 7.3.3    | Hiện tượng dương tính giả và âm tính giả . . . . .                  | 58        |
| <b>8</b> | <b>Tổng kết</b>                                                     | <b>59</b> |
| 8.1      | Kết quả đạt được trong quá trình hiện thực luận văn . . . . .       | 59        |
| 8.2      | Hạn chế và hướng phát triển trong tương lai . . . . .               | 59        |
| 8.2.1    | Hạn chế . . . . .                                                   | 59        |
| 8.2.2    | Hướng phát triển trong tương lai . . . . .                          | 60        |
|          | <b>Tài liệu tham khảo</b>                                           | <b>61</b> |
| <b>A</b> | <b>Phụ lục</b>                                                      | <b>63</b> |
| A.1      | Mã độc ngân hàng Ursnif năm 2019 . . . . .                          | 63        |
| A.2      | Mã độc ngân hàng Ursnif năm 2020 . . . . .                          | 64        |
| A.3      | Chiến dịch tấn công của tổ chức Gamaredon đầu năm 2022 . . . . .    | 64        |
| A.4      | Chiến dịch tấn công của tổ chức SideWinder năm 2019 . . . . .       | 65        |
| A.5      | Chiến dịch tấn công của tổ chức Bladabindi năm 2021 . . . . .       | 65        |

## Danh sách bảng

|     |                                                                    |    |
|-----|--------------------------------------------------------------------|----|
| 7.1 | Danh sách các mẫu đã được kiểm tra . . . . .                       | 54 |
| 7.2 | Danh sách các mẫu đã được kiểm tra (Tiếp theo) . . . . .           | 55 |
| 7.3 | Bảng tổng kết . . . . .                                            | 55 |
| 7.4 | Đánh giá độ chính xác của mô hình . . . . .                        | 56 |
| 7.5 | So sánh tiêu chí giữa công cụ đề xuất và các công cụ đã nghiên cứu | 56 |

## Danh sách hình vẽ

|     |                                                                                 |    |
|-----|---------------------------------------------------------------------------------|----|
| 1.1 | Thống kê các cuộc tấn công bằng Ransomware từ 01/2020 tới 10/2023 [3] . . . . . | 3  |
| 2.1 | Chương trình có <b>21.394</b> dòng . . . . .                                    | 11 |
| 2.2 | Chiến dịch dùng PowerShell làm Dropper . . . . .                                | 13 |
| 2.3 | Cấu trúc cơ bản của một file PE [5] . . . . .                                   | 14 |
| 2.4 | Cơ chế hoạt động của IAT . . . . .                                              | 21 |
| 2.5 | Cơ chế hoạt động của IAT Hooking . . . . .                                      | 21 |
| 2.6 | Cơ chế hoạt động của Inline Hooking . . . . .                                   | 22 |
| 3.1 | Kiểm tra hệ thống Virtual Machine . . . . .                                     | 29 |
| 4.1 | Tổng quan về ngôn ngữ được sử dụng trong hệ thống . . . . .                     | 31 |
| 6.2 | Cấu trúc của report.json . . . . .                                              | 45 |
| 6.3 | Sơ đồ hoạt động của quá trình Detector . . . . .                                | 46 |
| 6.4 | Sơ đồ hoạt động của quá trình CreateFile và OpenFile . . . . .                  | 47 |
| 6.5 | Sơ đồ hoạt động của quá trình WriteFile . . . . .                               | 48 |
| 6.6 | Sơ đồ hoạt động của quá trình ReadFile . . . . .                                | 49 |
| 6.1 | Cấu trúc đề xuất của công cụ . . . . .                                          | 52 |
| A.1 | Kịch bản tấn công . . . . .                                                     | 63 |
| A.2 | Kịch bản tấn công . . . . .                                                     | 64 |
| A.3 | Kịch bản tấn công . . . . .                                                     | 65 |
| A.4 | Kịch bản tấn công . . . . .                                                     | 65 |
| A.5 | Kịch bản tấn công . . . . .                                                     | 66 |

## Danh mục từ viết tắt

|      |                                    |
|------|------------------------------------|
| AMSI | Antimalware Scan Interface         |
| API  | Application Programming Interface  |
| APT  | Advanced Persistent Threat         |
| BKAV | Bách Khoa Antivirus                |
| CLI  | Command Line Interface             |
| CLR  | Common Language Runtime            |
| COFF | Common Object File Format          |
| DOS  | Disk Operating System              |
| DLL  | Dynamic Link Library               |
| IAT  | Import Address Table               |
| IP   | Internet Protocol                  |
| PID  | Process Identifier                 |
| PE   | Portable Executable                |
| PS   | PowerShell                         |
| RVA  | Relative Virtual Address           |
| VBA  | Visual Basic For Applications      |
| WMI  | Windows Management Instrumentation |

## 1.1 Giới thiệu đề tài

Cùng với sự phát triển mạnh mẽ của công nghệ thông tin và mạng Internet, sự tương tác giữa con người và máy tính ngày càng trở nên rộng rãi và quan trọng hơn bao giờ hết. Tất cả những hoạt động hằng ngày của con người, từ học tập, làm việc cho đến giải trí đều có sự tham gia của các thiết bị thông minh. Điều này đã tạo điều kiện cho các cuộc tấn công dưới hình thức mã độc (malware) nhắm vào người dùng ngày càng gia tăng không những về số lượng mà còn phức tạp hơn bao giờ hết. Theo báo cáo của Statista<sup>1</sup>, trong năm 2022, đã có tới 5.5 tỉ vụ tấn công bằng mã độc diễn ra trên toàn thế giới. Riêng ở Việt Nam, theo số liệu thống kê của BKAV năm 2022, thiệt hại do virus máy tính gây ra đối với người dùng Việt Nam ở mức 21.2 nghìn tỷ đồng<sup>2</sup>. Hơn 180.000 máy tính của các cơ quan, tổ chức Việt Nam bị nhiễm mã độc APT. Trong đó mã độc mã hóa tống tiền (Ransomware) ở Việt Nam có xu hướng tăng mạnh, chỉ riêng trong năm 2022 đã ghi nhận được hơn 57.000<sup>3</sup> cuộc tấn công đứng thứ ba Đông Nam Á chỉ xếp sau Indonesia và Thái Lan. Trước những tác hại to lớn đó, việc phát hiện và kiểm soát mã độc tống tiền để bảo vệ an toàn thông tin cho các tổ chức trên thế giới nói chung và Việt Nam nói riêng đang là một vấn đề rất được quan tâm.

Hiện nay, một phương pháp phổ biến để phát hiện mã độc được các phần mềm diệt virus sử dụng là so sánh file với mẫu mã độc đã được nhận diện từ trước. Khi có một file mới đi vào máy tính (thông qua sao chép từ USB hay tải về từ Internet) file này sẽ được lưu vào bộ nhớ thứ cấp, những phần mềm diệt virus sẽ quét các file này và so sánh chữ ký (Signature)<sup>4</sup> của file với một kho dữ liệu các Signature của những mã độc đã biết. Tiếp đó, file sẽ được quét để tìm kiếm một số dãy byte nhất định. Đây là những dãy byte luôn xuất hiện trong các mẫu mã độc cũ. Nếu Signature của file xuất hiện trong cơ sở dữ liệu mã độc, hay trong file có chứa một

<sup>1</sup><https://www.statista.com/statistics/873097/malware-attacks-per-year-worldwide/>

<sup>2</sup><https://www.bkav.com.vn/tin-tuc-noi-bat/-/view-content/1459709/tong-ket-an-ninh-mang-nam-2022-va-du-bao-2023>

<sup>3</sup><https://vneconomy.vn/viet-nam-xep-thu-3-dong-nam-a-voi-hon-57-000-cuoc-tan-cong-ransomware-trong-nam-2022.htm>

<sup>4</sup><https://www.sentinelone.com/blog/what-is-a-malware-file-signature-and-how-does-it-work/>

---

chuỗi byte đặc biệt như đã nêu ở trên, từ đó có thể kết luận được rằng đó là một file có chứa mã độc. Từ đó hệ thống sẽ cô lập và loại bỏ nó.

Phương pháp phát hiện mã độc dựa vào các đặc trưng được đề cập ở trên là một phương pháp rất hiệu quả. Tuy nhiên, phương pháp này vẫn còn tồn tại một số điểm yếu nhất định. Trước hết, nó chỉ nhận diện được các loại mã độc đã biết, những loại mã độc mới, chưa có trong cơ sở dữ liệu đều sẽ bị bỏ qua. Thêm nữa, đây là phương pháp hoạt động trên các file đã được lưu trữ trên bộ nhớ thứ cấp. Điều này có nghĩa là nó không thể nhận diện được những dạng mã độc tồn tại dưới dạng một chuỗi đã bị mã hóa (Obfuscation), hay những đoạn mã ẩn (Macro) [11] trong các file của các phần mềm văn phòng (Word, Docx, Excel, PowerPoint). Khi các loại file này được mở thì các đoạn mã độc ẩn bên trong sẽ được kích hoạt, lúc này thì hệ thống diệt virus mới phát hiện, cảnh báo và bắt đầu triển khai các biện pháp phòng vệ, tuy nhiên điều này được cho là quá trễ do mã độc đã xâm nhập vào hệ thống và mã độc sẽ tự hoạt động mỗi khi nạn nhân khởi động máy[17].

Cùng với sự phát triển về khoa học kỹ thuật dẫn đến các mã độc hiện nay ngày càng nguy hiểm, phức tạp hơn, khai thác những lỗ hổng chưa được phát hiện (Zero-Day)<sup>5</sup> trong các ứng dụng và sử dụng rất nhiều kỹ thuật tinh vi để lẩn tránh các giải pháp bảo mật. Một số kỹ thuật phổ biến hiện nay đang được sử dụng: ẩn náu dưới các phần mềm phổ biến như Google, Office, sử dụng kỹ thuật tàng hình (fileless), được lập trình bởi nhiều ngôn ngữ khác nhau, hoặc được triển khai qua nhiều bước để lẩn tránh các phần mềm diệt virus của hệ thống.

Trong đó kỹ thuật triển khai mã độc bằng ngôn ngữ Powershell ngày càng tăng cao. Bằng cách thực thi một đoạn chuỗi Powershell được ẩn trong các file giả mạo. Đa số mã độc được triển khai theo cách này thì rất khó bị phát hiện. Theo dữ liệu mới nhất (Hình 1.1) từ công ty an ninh mạng BlackFog<sup>6</sup>, PowerShell đã được sử dụng trong 48% cuộc tấn công Ransomware chỉ trong tháng 10 năm 2023.

Chính vì những lý do trên, đề tài này hướng đến việc xây dựng một công cụ có khả năng cô lập môi trường bằng kỹ thuật Sandbox, qua đó có thể ứng dụng Sandbox này trong việc phát hiện được hành vi của mã độc Ransomware được viết bằng Powershell. Công cụ sẽ được tập trung phát triển cho hệ điều hành Windows, hiện là một trong những hệ điều hành được sử dụng phổ biến nhất trên thế giới.

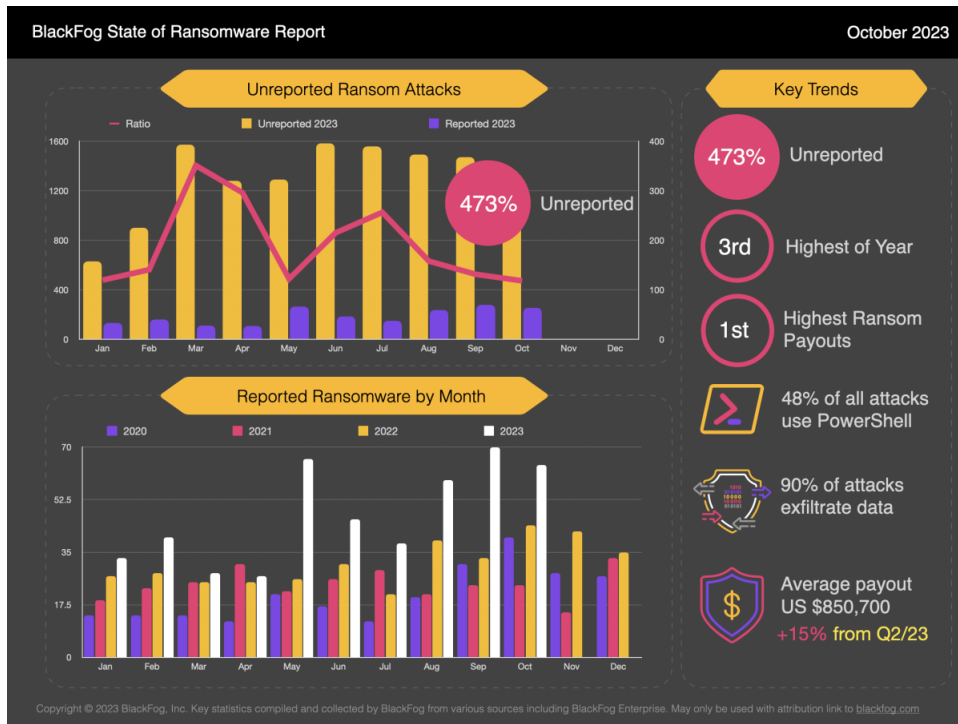
## 1.2 Mục tiêu và phạm vi của đề tài

Mục tiêu của đề tài luận văn này là nghiên cứu, hiểu và hiện thực được một công cụ có sử dụng kỹ thuật Sandbox để cô lập hoặc ngăn chặn quá trình thực thi của một chương trình PowerShell. Từ đó công cụ này có thể được ứng dụng trong việc phát hiện hành vi của mã độc Ransomware và hành vi của một số mã độc phân tán khác được viết bằng ngôn ngữ PowerShell. Công cụ có khả năng chạy được chương trình PowerShell dưới dạng một đoạn script PowerShell hoặc một file PowerShell.

---

<sup>5</sup><https://www.ibm.com/topics/zero-day>

<sup>6</sup><https://www.blackfog.com/>



Hình 1.1: Thống kê các cuộc tấn công bằng Ransomware từ 01/2020 tới 10/2023 [3]

Trong phạm vi luận văn này, chúng tôi sẽ không giải quyết hết tất cả các loại mã độc mà chỉ tập trung vào phân tích các dấu hiệu và nhận dạng hành vi của mã độc Ransomware. Công cụ của chúng tôi có thể chạy trực tiếp trên hệ điều hành Windows 10 và 11. Do hệ điều hành này phổ biến nhất trên thế giới và là mục tiêu bị tấn công nhiều nhất.

## 1.3 Cấu trúc luận văn

Cấu trúc của Luận văn được chia thành 8 chương sau:

- **Chương 1 – Giới thiệu**

Giới thiệu tổng quan về đề tài, tình hình chung về an toàn thông tin trên thế giới nói chung và Việt Nam nói riêng. Đề xuất hướng phát triển, mục tiêu cần đạt được trong quá trình hiện thực đề tài.

- **Chương 2 – Kiến thức nền tảng**

Trình bày các kiến thức chung về mã độc, phân loại và chi tiết về loại mã độc mà sinh viên đang nghiên cứu. Chi tiết về cấu trúc của một file thực thi, kỹ thuật chèn mã trong file thực thi và giới thiệu về Sandbox.

- **Chương 3 – Các công cụ và nghiên cứu liên quan**

Trình bày về các công cụ và các nghiên cứu liên quan đến đề tài.



---

- **Chương 4 – Đề xuất giải pháp**

Đề xuất giải pháp để có thể hiện thực đề tài. Thiết kế hệ thống và đưa ra một số tiêu chí đánh giá.

- **Chương 5 – Phân tích các hàm API**

Phân tích chi tiết về chức năng và tham số của các hàm API được sử dụng.

- **Chương 6 – Hiện thực công cụ**

Chi tiết về hệ thống và cách hệ thống hoạt động.

- **Chương 7 – Kiểm thử và Đánh giá công cụ**

Kiểm tra và đánh giá hoạt động của công cụ trên các mẫu mã độc thực tế.

- **Chương 8 – Tổng kết**

Tổng kết các kết quả đã đạt được trong quá trình nghiên cứu và phát triển luận văn, thảo luận những hạn chế và hướng phát triển trong tương lai.

## 2.1 Mã độc

Mã độc (Malware) hay còn được gọi với tên phần mềm độc hại (Malicious software) là một chương trình hoặc đoạn mã, thường được phân tán thông qua mạng, có khả năng lây nhiễm, gây hại cho hệ thống máy tính, đánh cắp dữ liệu hoặc thực hiện mọi hành vi mà kẻ tấn công mong muốn. Bất kỳ một phần mềm nào tác động và phá vỡ đến ba yếu tố cơ bản trong hệ thống an toàn thông tin: tính bí mật (Confidentiality), tính toàn vẹn (Integrity) và tính khả dụng (Availability) của dữ liệu người dùng, máy tính hoặc môi trường mạng đều có thể được xem như mã độc. Hầu hết mọi người sử dụng máy tính đều gọi chung các chương trình này là Virus nhưng thực chất mã độc có rất nhiều loại khác nhau. Mặc dù đa dạng về loại hình và phương thức tấn công, phần mềm độc hại thường có một trong các mục tiêu sau:

- Gửi thư rác từ máy bị nhiễm tới các mục tiêu khác.
- Lấy cắp dữ liệu nhạy cảm từ máy bị nhiễm.
- Tạo quyền truy cập từ xa đến máy mục tiêu.
- Ngụy trang và nghe lén.
- Lấy thông tin mạng nội bộ.

### 2.1.1 Một số loại mã độc

Hiện nay với tốc độ phát triển rất nhanh của công nghệ, dẫn đến xuất hiện các loại mã độc mới nguy hiểm hơn, ẩn náu tinh vi nên rất khó bị phát hiện. Tuy nhiên chúng tôi chỉ giới thiệu một số loại mã độc phổ biến hiện nay:

- **Dropper** là một mã dạng mã độc được thiết kế để triển khai một chương trình mã độc khác. Mã độc dạng này thường được viết bằng ngôn ngữ scripting nên có kích thước rất nhỏ. Và bằng nhiều cách khác nhau mà có thể vượt qua được cơ chế bảo vệ của hệ thống.

- 
- **Stealer** được thiết kế để đánh cắp thông tin nhạy cảm từ hệ thống nạn nhân. Mục tiêu chính là thu thập dữ liệu như thông tin đăng nhập, mật khẩu, thông tin tài chính, dữ liệu cá nhân, và các thông tin nhạy cảm khác mà nó có thể khai thác để thực hiện các hoạt động phi pháp như đánh cắp danh tính, hoặc bán thông tin trên các chợ đen trực tuyến.
  - **Spyware** là một phần mềm gián điệp theo dõi hoạt động, thu thập thông tin và dữ liệu quan trọng trên thiết bị nạn nhân.
  - **Rootkit** là tập hợp của các file được cài đặt lên hệ thống nhằm biến đổi các chức năng chuẩn của hệ thống thành các chức năng tiềm ẩn các tấn công nguy hiểm.
  - **Adware** là mã độc theo dõi các ứng dụng trình duyệt, lịch sử tải xuống, hiển thị hoặc biểu ngữ thu hút nạn nhân.
  - **Ransomware** là mã độc mã hóa dữ liệu trong máy tính của nạn nhân, khiến dữ liệu của nạn nhân không thể truy cập được cho đến khi nhận được khoản tiền chuộc để đổi lấy dữ liệu.
  - **Keylogger** là mã độc theo dõi mọi hoạt động trên máy tính từ các tổ hợp phím. Có thể ghi lại nội dung email, password, nội dung nhạy cảm, ...
  - **Backdoor** tạo một kết nối tới máy mục tiêu thông qua các giao thức TCP hoặc UDP, thực hiện quyền truy cập từ xa nhằm đưa các tác nhân gây hại khác vào thiết bị.

Ngoài ra còn có vô số loại mã độc khác như: BotNet, Trojan, Scareware, Fileless Malware, ...

### 2.1.2 Ransomware

**Ransomware** là một loại phần mềm độc hại được sử dụng với mục đích tống tiền người dùng. Bằng cách mã hóa một số file dữ liệu quan trọng khiến dữ liệu của nạn nhân không thể truy cập được cho đến khi nhận được khoản thanh toán tiền chuộc. Hiện nay các cuộc tấn công Ransomware diễn ra ngày càng nguy hiểm và phức tạp hơn, gây thiệt hại rất lớn không chỉ đến người dùng mà còn cho các doanh nghiệp và tổ chức chính phủ. Trong năm 2024, Việt Nam đã có hai cuộc tấn công Ransomware lớn vào hai công ty VNDirect<sup>1</sup> và PVOil<sup>2</sup> để lại thiệt hại rất lớn. Với trường hợp của VNDirect, bên kẻ tấn công đã có sai lầm nên các chuyên gia có thể giải mã mà không cần tiền chuộc, tuy nhiên, không phải mọi tấn công đều bị sai phạm như thế và thường không có chuyên gia phân tích chuyên sâu.

Bắt đầu từ cách thức tấn công, Ransomware thường lợi dụng các kênh truyền thông trực tuyến như email, trang web độc hại, hoặc thậm chí là các ứng dụng giả

---

<sup>1</sup><https://vietnamnet.vn/vndirect-bi-tan-cong-ransomware-nguy-hiem-nhu-the-nao-2265375.html>

<sup>2</sup><https://antoanthongtin.vn/hacker-malware/he-thong-cong-nghe-thong-tin-cua-pvoil-bi-tan-cong-ransomware-109941>

---

mạo để xâm nhập vào hệ thống của nạn nhân. Một khi đã tiếp cận được hệ thống, nó sẽ bắt đầu quá trình mã hóa các tập tin dữ liệu quan trọng.

Nhưng tác động của Ransomware không chỉ dừng lại ở việc mã hóa dữ liệu. Nó còn gửi thông tin về hệ thống của nạn nhân về một máy chủ kiểm soát (Command and Control Server - C&Cs)<sup>3</sup>, tạo điều kiện cho kẻ tấn công có thể kiểm soát từ xa và yêu cầu khoản tiền chuộc. Khi nạn nhân bị khóa hệ thống và dữ liệu của họ bị mất điều quan trọng, thông điệp hiển thị trên màn hình thường là yêu cầu thanh toán tiền chuộc trong một khoảng thời gian nhất định để nhận lại khóa giải mã.

Hầu hết các loại mã độc tống tiền đều có một số hành vi chính như sau: khóa hệ thống, mã hóa dữ liệu, xóa dữ liệu, lấy dữ liệu và gửi một thông báo trên màn hình hệ thống[2].

- Khóa hệ thống: bao gồm việc từ chối truy cập vào thiết bị, dữ liệu ...
- Mã hóa dữ liệu: mã hóa các tệp dữ liệu quan trọng trong hệ thống. Mục đích là để người bị tấn công phải trả tiền.
- Xóa dữ liệu: một vài loại Ransomware sẽ xóa toàn dữ liệu bất kể người dùng có trả tiền hay không.
- Lấy dữ liệu: lấy thông tin nhạy cảm của người dùng. Các thông tin này có thể công khai trên các trang web hoặc bán lại cho bên thứ ba.
- Thông báo: hầu hết các loại mã độc tống tiền sẽ luôn hiển thị một thông báo trên màn hình yêu cầu họ phải trả tiền để lấy lại dữ liệu.

### 2.1.3 Fileless Malware

**Fileless Malware** là một loại chương trình độc hại được triển khai dưới dạng những đoạn script mà không phải là những file thực thi (executable file) thông thường[15]. Nó tận dụng các công cụ và những chức năng có sẵn trong hệ thống để thực hiện các hoạt động độc hại. Các kỹ thuật Fileless Malware thường tập trung vào việc sử dụng bộ nhớ hệ thống, script, hay các ứng dụng có sẵn để thực hiện các tác vụ độc hại.

Các kỹ thuật của Fileless Malware thường tập trung vào việc sử dụng các thành phần có sẵn trong hệ thống để tiến hành các hoạt động độc hại. Một số phương pháp phổ biến bao gồm:

- **Memory-Resident Malware** (sử dụng vùng nhớ): mã độc tấn công và hoạt động trên vùng nhớ của một tiến trình khác.
- **Script-Based Fileless Malware** (sử dụng các công cụ mặc định): sử dụng các ngôn ngữ Scripting như PowerShell, JavaScript, hay VBScript để thực hiện các tác vụ độc hại.

---

<sup>3</sup><https://www.trendmicro.com/vinfo/us/security/definition/command-and-control-server>

---

Để tránh bị phát hiện bởi các hệ thống diệt virus, mã độc sẽ được đính kèm vào trong các tài liệu văn phòng, trong các ứng dụng hay trong các đường link quảng cáo. Dưới đây là một số kỹ thuật tàng hình được sử dụng phổ biến:

- **Malicious Document** (kỹ thuật ẩn trong tệp tài liệu): tận dụng các lỗ hổng bảo mật mã độc được giấu vào trong các file tài liệu (PDF), hay trong các file ứng dụng văn phòng (Excel, Word, ...).
- **Malicious Script** (kỹ thuật ẩn trong các đoạn script): mã độc dạng này được viết bằng ngôn ngữ Scripting, có thể thực thi trực tiếp trên môi trường dòng lệnh hoặc PowerShell mà không cần biên dịch xuống mã máy. Loại này thường được đính kèm trong các link từ các trang web độc hại.
- **Living off the Land** (kỹ thuật sử dụng công cụ của hệ thống): sử dụng các công cụ và chức năng có sẵn trên hệ thống, như PowerShell, WMI (Windows Management Instrumentation), hay công cụ quản lý hệ thống, để thực hiện các hoạt động độc hại mà không cần tải về mã độc hại từ bên ngoài.
- **Malicious Code in Memory** (kỹ thuật chèn code vào vùng nhớ): mã độc chèn vào trong vùng nhớ của một tiến trình khác và thực thi ngầm.

## 2.2 PowerShell

**PowerShell** hay đầy đủ là Windows PowerShell là một tiện ích dòng lệnh và ngôn ngữ kịch bản, một công cụ dựa trên giao diện dòng lệnh (Command Line Interface, gọi tắt là CLI) cho phép các nhà phát triển, quản trị viên có thể tự động hóa các tác vụ và cấu hình bằng cách sử dụng lệnh.

Đây là một công cụ được phát triển dựa trên **DotNET Framework**<sup>4</sup> dành cho các quản trị viên công nghệ thông tin nhằm quản lý các tác vụ, cho phép tương tác với dịch vụ và chức năng của hệ điều hành, mang lại sự tiện lợi, hiệu quả trong việc viết và quản lý các tập lệnh, kịch bản. PowerShell được triển khai dựa trên dịch vụ độc quyền của Windows. Sau đó được Windows cung cấp mã nguồn mở và hoạt động trên nhiều nền tảng khác nhau như Linux và macOS. PowerShell đã được cài đặt mặc định trên nền tảng Windows từ phiên bản Windows 7 trở đi.

Một số tính năng mạnh mẽ của PowerShell:

- **Object-Oriented** (hướng đối tượng): PowerShell không chỉ là một dòng lệnh, mà còn là một môi trường lập trình với hỗ trợ đầy đủ cho đối tượng (object-oriented). Mọi thứ trong PowerShell đều là đối tượng, giúp dễ dàng xử lý và chuyển đổi dữ liệu.
- **Remoting** (điều khiển từ xa): cho phép thực hiện các lệnh và các đoạn script trên các máy tính từ xa.
- **Task Automation** (tự động các tác vụ): cho phép tự động hóa nhiều công việc quản lý hệ thống, từ cài đặt ứng dụng đến quản lý tác vụ hàng ngày.

---

<sup>4</sup><https://dotnet.microsoft.com/en-us/>

- **Scripting Language:** có thể chạy mà không cần phải biên dịch (compile) sang mã máy.
- **Extensibility** (khả năng mở rộng): cho phép mở rộng bằng cách tạo các mô-đun (modules).
- **Quản lý cấu hình và hệ thống:** quản lý cấu hình hệ thống, quản lý dịch vụ, lên lịch thực thi các tác vụ, sự kiện, định tuyến, quyền và nhiều hơn nữa.
- **Tích hợp với Windows:** là một phần của hệ điều hành Windows, điều này có nghĩa rằng nó được tích hợp sâu vào môi trường Windows và có khả năng tương tác với hầu hết các thành phần hệ thống Windows.

## 2.2.1 PowerShell Obfuscation

Để tránh bị phát hiện, kẻ tấn công thường sử dụng một số mã kỹ thuật làm rối mã nguồn (Obfuscation) [13][8] nhằm mục đích làm cho mã khó hiểu đối với cả phần mềm diệt virus và các chuyên gia phân tích. Về mặt hình thức, sự xáo trộn có thể được xem là sự thay đổi về mã nguồn, tuy nhiên về mặt ngữ nghĩa (semantic) và hành vi (behavior) của chương trình vẫn không thay đổi.

Để người đọc có thể hiểu hơn về kỹ thuật làm rối mã nguồn, chúng tôi thực hiện làm tối trên đoạn mã 2.1, thông qua các kỹ thuật làm rối mã khác nhau, ta có được các đoạn mã 2.2, 2.3, 2.4, 2.5, 2.6, 2.7 và 2.8.

```
1 Start-Process malware.exe
```

Code 2.1: Chương trình mã độc

**String-based** mã được xử lý dưới dạng một chuỗi, áp dụng các thao tác liên quan như nối, sắp xếp lại, đảo ngược hoặc thay thế chuỗi con. Sau đó mã sẽ được khởi chạy như một câu lệnh trong chương trình thông qua lệnh **Invoke-Expression** hoặc toán tử "&".

Ví dụ: hai đoạn chương trình bên dưới (Chương trình 2.2 và 2.3) bị tách thành nhiều chuỗi con. Ở ví dụ đầu thì các chuỗi con được sắp xếp một cách ngẫu nhiên, khiến cho việc phân tích trở nên rất khó khăn, chuỗi được khôi phục thông qua định dạng chuỗi (Format String) trong PowerShell. Ở ví dụ thứ hai thì thứ tự các chuỗi con không thay đổi và dễ dàng khôi phục thông qua toán tử nối chuỗi đơn giản "+". Thông qua lệnh Invoke-Expression thì chuỗi đó sẽ được thực thi. Ở dạng này thì kỹ thuật thay đổi thứ tự chuỗi con sẽ mang lại hiệu quả hơn so với kỹ thuật còn lại nếu đoạn chương trình lớn hơn.

```
1 (
2     "{2}{5}{1}{3}{4}{0}" -f "xe","Pro","St","cess mal","ware.e","
   art-"
3 ) | Invoke-Expression
```

Code 2.2: Kỹ thuật thay đổi thứ tự chuỗi con

```
1 ( "S"+"tart-Process "+"mal"+"ware.exe" ) | Invoke-Expression
```

Code 2.3: Kỹ thuật nối chuỗi

**Base64** thực thi mã dưới dạng chuỗi đã bị mã hóa bằng thuật toán base64.

Ví dụ: Để thực thi, chuỗi mã hóa phải được chuyển làm đầu vào cho PowerShell và cờ “-e” (-EncodedCommand). Cờ này làm cho PowerShell biết rằng nó phải giải mã chuỗi trước khi thực thi.

```
1 powershell -e "U3RhcnQtUHJvY2VzcyBtYWx3YXJlLmV4ZQ=="
```

Code 2.4: Kỹ thuật thực thi chuỗi base64

**Encoded** kiểu che giấu này được thực hiện bằng cách chuyển đổi từng ký tự riêng lẻ thành ký tự trùng khớp của một cột trên bảng ASCII <sup>5</sup> hoặc bằng cách áp dụng một số thuật toán phổ biến. Kết quả có thể được thực thi thông qua lệnh Invoke-Expression.

Ví dụ: đoạn chương trình gốc đã bị ẩn thông qua thuật toán **XOR** với khóa là 0x42 (Chương trình 2.5).

```
1 (
2     "17_54_35_48_54_111_18_48_45_33_39_49_49_98_47_35_46_53_35_48_
3     39_108_39_58_39" -split "_" | ForEach-Object {
4         [char]( $_ -bxor "0x42" )
5     }
6 ) -join "" | Invoke-Expression
```

Code 2.5: Kỹ thuật mã hóa bằng thuật toán XOR

**Compressed** mã độc sẽ được tạo ra thông qua việc áp dụng một thuật toán nén dữ liệu mà PowerShell hỗ trợ.

Ví dụ: đoạn chương trình 2.1 được được nén lại bằng thuật toán ZIP, và được lưu trữ dưới dạng base64. Đoạn chương trình 2.6 sẽ thực hiện giải nén ra vào thời gian thực thi lệnh.

- Dòng 1 đoạn chương trình được mã hóa được lưu vào biến *\$body*.
- Dòng 5 giải mã *\$body* thông qua thuật toán base64 và lưu vào biến *\$zipBytes*.
- Dòng 6-9 khởi tạo biến *\$zipStream* dùng để lưu vùng nhớ chứa dữ liệu được giải nén trong biến *\$zipBytes* bằng đối tượng IO.Compression.DeflateStream với chế độ *Decompress*.
- Dòng 10 tạo biến *\$scriptBin* để chứa dữ liệu.
- Dòng 11 đọc dữ liệu từ *\$zipStream* và lưu vào biến *\$scriptBin*.
- Dòng 12 đưa dữ liệu bytes trong *\$scriptBin* về kiểu chuỗi cho biến *\$script*.
- Dòng 13 thực thi *\$script* bằng hàm IEX (Invoke-Expression).

```
1 $body = "UESDBBQAAAAIAE2FjFeFLf2mIwAAABkAAAAIAAAAZm1sZS50eHQ
2 FQEEJACAQq2IBL4tggiH7KcLuQOOPWVD1obuY2Q72gxj8NFBLAQIUABQAAAA
3 IAE2FjFeFLf2mIwAAABkAAAAIAAAAAAAAAAAAAAAAAAAAAABmaWx1LnR4dFB
4 LBQYAAAAAAQABADYAAABJAAAAAA="
```

<sup>5</sup><https://upload.wikimedia.org/wikipedia/commons/d/dd/ASCII-Table.svg>

```

5 $zipBytes = [System.Convert]::FromBase64String($body)
6 $zipStream = New-Object IO.Compression.DeflateStream(
7     [IO.MemoryStream][Byte[]]$zipBytes,
8     [IO.Compression.CompressionMode]::Decompress
9 )
10 $scriptBin = New-Object Byte[]($zipBytes.Length)
11 $zipStream.Read($scriptBin, 0, $scriptBin.Length) | Out-Null
12 $script = [System.Text.Encoding]::UTF8.GetString($scriptBin)
13 IEX $script

```

Code 2.6: Kỹ thuật Compressed

**Randomization:** hình thức này bao gồm việc chèn ngẫu nhiên các ký tự in hoa, ký tự khoảng trắng, hoặc các ký hiệu không được diễn giải bởi PowerShell.

Ví dụ: các ký tự trong chương trình 2.7 bao gồm chữ hoa và chữ thường xen kẽ nhau, bên cạnh đó còn xuất hiện thêm các ký tự “`”, các ký tự đặc biệt này sẽ bị PowerShell phớt lờ trong quá trình thực thi. Bên cạnh đó cú pháp của PowerShell không phân biệt chữ hoa với chữ thường.

```

1 powershell "sTArT-p`ROC`ESS malware.exe"

```

Code 2.7: Kỹ thuật chèn các ký tự ngẫu nhiên

**Token Manipulation:** chia chương trình thành các chuỗi con lưu vào nhiều biến khác nhau.

Ví dụ: đoạn chương trình 2.8 được chia thành nhiều biến khác nhau, mỗi biến sẽ lưu một chuỗi chương trình con. Bằng kỹ thuật nối chuỗi logic bên dưới thì có thể khôi phục lại đoạn chương trình gốc và thực thi nó thông qua qua hàm IEX.

```

1 $a = "S"
2 $b = "tart-"
3 $c = "Process "
4 $d = "mal"
5 $e = "ware."
6 $f = "exe"
7 IEX "$a$b$c$d$e$f"

```

Code 2.8: Kỹ thuật Token Manipulation

**Mở rộng code:** kỹ thuật này được thực hiện bằng cách chèn những đoạn chương trình vô nghĩa nhưng không ảnh hưởng đến hành vi của mã độc, điều này làm tăng kích thước của chương trình.

Ví dụ: xét mẫu mã độc PowerShell Ransomware có mã sha256 (Hình 2.1)  
**c597c75c6b6b283e3b5c8caeee095d60902e7396536444b59513677a94667ff8**

length : 1,274,621 lines : 21,394

Hình 2.1: Chương trình có **21.394** dòng

Từ các ví dụ trên có thể thấy được việc phân tích một PowerShell Script là không hề đơn giản. Tuy nhiên trong thực tế, khi mã độc PowerShell được triển khai thì các kỹ thuật Obfuscation sẽ được sử dụng cùng nhau để có thể đạt được



---

hiệu quả cao, chính vì vậy việc phân tích tĩnh sẽ gây ra rất nhiều khó khăn đối với các chuyên gia cũng như các ứng dụng diệt virus nhận dạng bằng phương pháp kiểm tra Signature<sup>6</sup>. Ngoài các kỹ thuật được nêu trên còn có thể có những kỹ thuật Obfuscation khác mà chưa được biết đến.

### 2.2.2 Ransomware sử dụng Fileless PowerShell làm Dropper Malware

Trong bối cảnh công nghệ ngày càng phát triển, các hệ thống Anti-Virus trên máy tính cũng trở nên mạnh mẽ và tinh vi hơn. Các phương pháp tấn công truyền thống thông qua các tệp tin thực thi (.exe, .dll) ngày càng trở nên kém hiệu quả, bởi chúng dễ dàng bị phát hiện bởi các Signature của phần mềm bảo mật. Do đó, để vượt qua hàng rào bảo vệ này, kẻ tấn công ngày nay đang chuyển hướng sang các kỹ thuật tấn công tinh vi hơn, bằng cách sử dụng mã độc dạng Dropper<sup>7</sup> làm giai đoạn đầu tiên cho một chuỗi các tấn công phức tạp hơn.

Bằng cách sử dụng ngôn ngữ PowerShell và kỹ thuật Obfuscation (đã được nêu ở mục trên), Dropper sẽ dễ dàng vượt qua được hệ thống anti-virus do ngôn ngữ PowerShell rất khó để có thể phát hiện bằng Signature. Khi đã vào được hệ thống và kích hoạt thì Dropper có thể tải xuống và cài đặt nhiều loại mã độc khác nhau. Khi đã tải xuống thành công thì mã độc PowerShell này là có thể thực thi động chương trình trong vùng nhớ của nó mà không cần tạo ra bất kỳ tệp tin thực thi nào trên hệ thống. Việc này giúp mã độc tránh bị phát hiện bởi các hệ thống chống virus, vì không có tệp tin nào bị nghi ngờ để quét.

Dưới đây là một dạng Dropper sử dụng PowerShell (Hình 2.2) được sử dụng trong thời điểm bùng phát COVID-19<sup>8</sup>. Trong chiến dịch này PowerShell đã tải xuống và thực thi một file DotNet.

## 2.3 Các kỹ thuật phân tích mã độc

Phân tích mã độc là quá trình nghiên cứu và hiểu rõ về cách một chương trình hoạt động. Điều này có thể bao gồm phân tích cấu trúc của chương trình, việc tìm hiểu về các yếu tố chức năng và logic được sử dụng trong chương trình. Quá trình phân tích thường sẽ bao gồm hai dạng sau: phân tích tĩnh và phân tích động.

### 2.3.1 Phân tích tĩnh

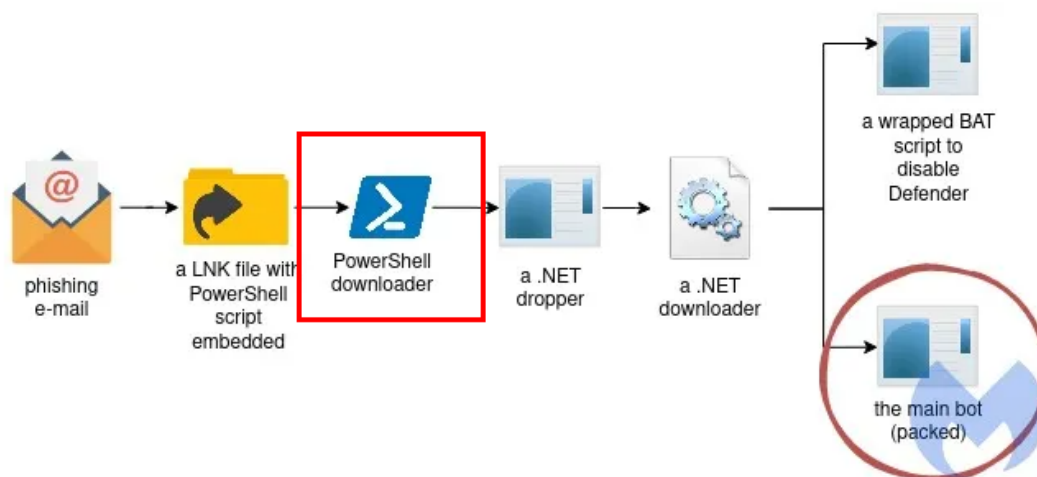
Đây là quá trình nghiên cứu và hiểu về các tính năng, hành vi, và cấu trúc của mã độc mà không cần thực thi nó trên một hệ thống thực tế. Kỹ thuật phân tích tĩnh thường được thực hiện bằng cách sử dụng các công cụ tự động để phân

---

<sup>6</sup>Signature là những đoạn mã, chuỗi byte, hoặc hành vi cụ thể của mã độc đã biết

<sup>7</sup>Dropper và Downloader là hai từ để chỉ một loại mã độc có hành vi giống nhau. Hành vi chính là triển khai một mã độc khác bằng cách tải xuống từ Internet

<sup>8</sup><https://www.malwarebytes.com/blog/threat-intelligence/2021/04/a-deep-dive-into-saint-bot-downloader>



Hình 2.2: Chiến dịch dùng PowerShell làm Dropper

tích, dịch ngược lại mã nguồn của một chương trình và từ đó có tìm hiểu logic của chương trình đó. Tìm kiếm và phân tích các tên miền (Domain), các chuỗi kí tự quan trọng, thuật toán được sử dụng trong chương trình. Tuy nhiên quá trình này vẫn còn nhiều hạn chế, nếu như chương trình đã bị Obfuscation, các chuỗi kí tự đã bị mã hóa, ..., dẫn đến quá trình phân tích tĩnh sẽ gặp rất nhiều khó khăn và tốn rất nhiều thời gian. Lúc này thì kỹ thuật phân tích động sẽ có hiệu quả hơn.

### 2.3.2 Phân tích động

Khác với kỹ thuật phân tích tĩnh chỉ tập trung vào phân tích mã nguồn của một chương trình thì kỹ thuật phân tích động lại tập trung vào phân tích hành vi của chương trình. Bằng cách cho chương trình chạy trực tiếp trên hệ thống rồi quan sát sự thay đổi trong hệ thống từ đó có thể biết được hành vi của chương trình. Tuy nhiên trước khi thực thi thì cần phải thiết lập một môi trường an toàn, cô lập chương trình trong một môi trường ảo để mã độc có thể thực thi và sử dụng các công cụ cho phép quan sát các hành động của nó: kiểm tra sự thay đổi của các file, folder trong hệ thống; kiểm tra kết nối Internet,...

## 2.4 Định dạng file Portable Executable

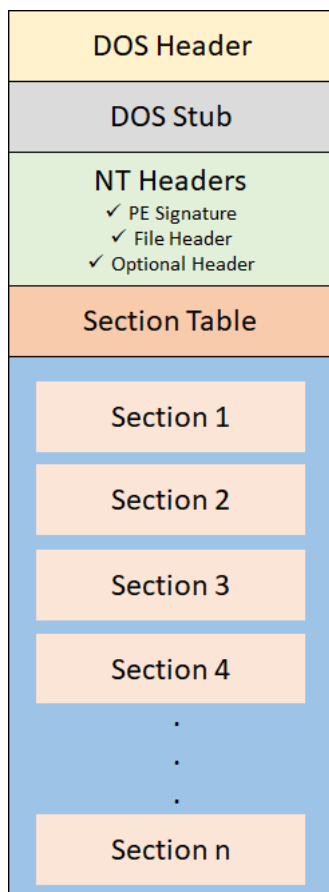
**Portable Executable (PE)** là định dạng file thực thi của hệ điều hành Windows. Định dạng là một cấu trúc dữ liệu trong đó chứa các thông tin cần thiết để bộ nạp (loader)<sup>9</sup> của Windows có thể quản lý và thực thi được các lệnh trong đó, được dựa trên cấu trúc của file COFF (Common Object File Format) [20]. Thường các file thực thi được trên Windows đều sử dụng định dạng PE bao gồm: Executable (.exe), (.dll). Ngoài ra còn một số file cũng có định dạng này như Kernel modules (.srv), Control panel applications (.cpl), ...

<sup>9</sup><https://www.techopedia.com/definition/8104/loader>

File PE có cấu trúc dữ liệu rất phức tạp, chứa các thông tin cần thiết để loader của hệ điều hành có thể tải vào bộ nhớ và thực thi nó. Do đó trong mục này chỉ có một số phần quan trọng của file PE liên quan đến đề án sẽ được liệt kê và mô tả.

### 2.4.1 Cấu trúc cơ bản

Cấu trúc chung của một file PE được minh họa ở hình 2.3.



Hình 2.3: Cấu trúc cơ bản của một file PE [5]

Một cấu trúc cơ bản gồm bốn phần sau: DOS Header, DOS Stub, NT Headers, Section Table. Ở hai biên bản PE32 (bộ xử lý 32-bit) cũng như PE32+ (bộ xử lý 64-bit) thì kích thước của cấu trúc DOS Header và DOS Stub đều như nhau.

### 2.4.2 DOS Header

Thường các file PE đều bắt đầu bằng **DOS Header**, vùng này chiếm giữ 64 bytes đầu tiên của file. Nó chứa các thông tin cơ bản về cấu trúc và quy định của file PE.

Cấu trúc cơ bản của một `_IMAGE_DOS_HEADER` được thể hiện như sau.

```
1 typedef struct _IMAGE_DOS_HEADER {
```

```

2      WORD    e_magic;
3      WORD    e_cblp;
4      WORD    e_cp;
5      WORD    e_crlc;
6      WORD    e_cparhdr;
7      WORD    e_minalloc;
8      WORD    e_maxalloc;
9      WORD    e_ss;
10     WORD    e_sp;
11     WORD    e_csum;
12     WORD    e_ip;
13     WORD    e_cs;
14     WORD    e_lfarlc;
15     WORD    e_ovno;
16     WORD    e_res[4];
17     WORD    e_oemid;
18     WORD    e_oeminfo;
19     WORD    e_res2[10];
20     LONG    e_lfanew;
21 } IMAGE_DOS_HEADER, *PIMAGE_DOS_HEADER;

```

Code 2.9: Cấu trúc của `_IMAGE_DOS_HEADER`

Cấu trúc này đóng vai trò quan trọng tới PE loader trên hệ điều hành MS-DOS, tuy nhiên chỉ có một vài giá trị thực sự quan trọng với PE loader trên hệ điều hành Windows, vì vậy chúng tôi sẽ không đề cập hết tất cả giá trị mà chỉ tập trung vào hai phần quan trọng là `e_magic`, `e_lfanew`:

- `e_magic` là giá trị đầu tiên trong cấu trúc DOS Header, chiếm hai bytes dữ liệu. Đây là một giá trị cố định **0x5A4D** hoặc **MZ** trong mã ASCII. Hệ điều hành sẽ kiểm tra xem giá trị này có tồn tại hay không để xác định xem đây có phải là một file thực thi trên Windows hay không.
- `e_lfanew` là giá trị cuối cùng trong cấu trúc DOS Header, chứa địa chỉ trỏ tới vùng NT Headers (sẽ được mô tả ở tiểu mục 2.4.4).

### 2.4.3 DOS Stub

**DOS Stub** thường bắt đầu từ vị trí `0x40` trong file PE và có kích thước 64 bytes nhưng lại không cố định mà tùy vào từng loại file PE. Vùng này chứa thông tin là một đoạn chương trình nhỏ thông báo rằng file không tương thích với hệ điều hành DOS [14] và sau đó kết thúc. Ví dụ: đoạn thông báo thường xuất hiện trong các file PE là **"This program cannot be run in DOS mode"**.

### 2.4.4 NT Headers

**NT Headers** hay còn được gọi với tên khác là **PE Headers**. Tên gọi này là thuật ngữ chung đại diện cho một cấu trúc được đặt tên là **IMAGE\_NT\_HEADERS**. Cấu trúc này gồm có ba thông tin quan trọng được sử dụng bởi loader: PE Signature, File Header, Optional Header. Cấu trúc này có sự khác nhau giữa hai phiên bản 32-bit và 64-bit.

Dưới đây là cấu trúc của `_IMAGE_NT_HEADERS` ở hai phiên bản.

```
1  typedef struct _IMAGE_NT_HEADERS {
2      DWORD Signature;
3      IMAGE_FILE_HEADER FileHeader;
4      IMAGE_OPTIONAL_HEADER32 OptionalHeader;
5  } IMAGE_NT_HEADERS32, *PIMAGE_NT_HEADERS32;
6
7  typedef struct _IMAGE_NT_HEADERS64 {
8      DWORD Signature;
9      IMAGE_FILE_HEADER FileHeader;
10     IMAGE_OPTIONAL_HEADER64 OptionalHeader;
11 } IMAGE_NT_HEADERS64, *PIMAGE_NT_HEADERS64;
```

Code 2.10: Cấu trúc của `_IMAGE_NT_HEADERS`

**Signature** là một DWORD chứa 4 bytes giá trị như sau 50h, 45h, 00h, 00h. Nó luôn luôn là một giá trị cố định ở dạng thập lục phân (hexadecimal) 0x50450000 tương ứng với chuỗi “PE00” trong mã ASCII.

**FileHeader** bao gồm 20 bytes tiếp theo của PE file, nó chứa các thông tin về sơ đồ bố trí vật lý và các đặc tính của file. Cấu trúc của phần này được định nghĩa như sau.

```
1  typedef struct _IMAGE_FILE_HEADER {
2      WORD Machine;
3      WORD NumberOfSections;
4      DWORD TimeDateStamp;
5      DWORD PointerToSymbolTable;
6      DWORD NumberOfSymbols;
7      WORD SizeOfOptionalHeader;
8      WORD Characteristics;
9  } IMAGE_FILE_HEADER, *PIMAGE_FILE_HEADER;
```

Code 2.11: Cấu trúc của `_IMAGE_FILE_HEADER`

- **NumberOfSections** số lượng các phân vùng (sections) được sử dụng trong file. Mỗi section sẽ có một vai trò khác nhau, ví dụ: vùng chứa mã thực thi (code section), vùng chứa dữ liệu (data section) và một số vùng khác.

**OptionalHeader** đây là thông tin quan trọng nhất trong cấu trúc NT Headers được tạo thành bởi 224 bytes (PE32) hay 240 bytes (PE32+) tiếp theo trong file. Nó chứa thông tin về sơ đồ Logic bên trong của một file PE. Loader của hệ điều hành sẽ tìm kiếm các thông tin trong cấu trúc này để tải và thực thi chương trình. Một số thông tin cụ thể bao gồm: địa chỉ bắt đầu (AddressOfEntryPoint), kích thước code (SizeOfCode), ...

Kích thước ở phần này khác nhau giữa hai phiên bản PE32 và PE32+ là do:

- Phiên bản 64-bit có 30 giá trị, còn phiên bản 32-bit có 31 giá trị do có thêm một giá trị tên BaseOfData kiểu DWORD theo ngay sau BaseOfCode.
- Các giá trị ImageBase, SizeOfStackReserve, SizeOfStackCommit, SizeOfHeapReserve, SizeOfHeapCommit có kiểu DWORD ở bản 32-bit và có giá trị ULONGLONG ở bản 64-bit.

Cấu trúc của một `_IMAGE_OPTIONAL_HEADER32` được thể hiện như sau.

```
1  typedef struct _IMAGE_OPTIONAL_HEADER {
2      WORD          Magic;
3      BYTE          MajorLinkerVersion;
4      BYTE          MinorLinkerVersion;
5      DWORD         SizeOfCode;
6      DWORD         SizeOfInitializedData;
7      DWORD         SizeOfUninitializedData;
8      DWORD         AddressOfEntryPoint;
9      DWORD         BaseOfCode;
10     DWORD         BaseOfData;
11     DWORD         ImageBase;
12     DWORD         SectionAlignment;
13     DWORD         FileAlignment;
14     WORD          MajorOperatingSystemVersion;
15     WORD          MinorOperatingSystemVersion;
16     WORD          MajorImageVersion;
17     WORD          MinorImageVersion;
18     WORD          MajorSubsystemVersion;
19     WORD          MinorSubsystemVersion;
20     DWORD         Win32VersionValue;
21     DWORD         SizeOfImage;
22     DWORD         SizeOfHeaders;
23     DWORD         CheckSum;
24     WORD          Subsystem;
25     WORD          DllCharacteristics;
26     DWORD         SizeOfStackReserve;
27     DWORD         SizeOfStackCommit;
28     DWORD         SizeOfHeapReserve;
29     DWORD         SizeOfHeapCommit;
30     DWORD         LoaderFlags;
31     DWORD         NumberOfRvaAndSizes;
32     IMAGE_DATA_DIRECTORY DataDirectory[16];
33 } IMAGE_OPTIONAL_HEADER32, *PIMAGE_OPTIONAL_HEADER32;
34 #define IMAGE_NUMBEROF_DIRECTORY_ENTRIES    16
```

Code 2.12: Cấu trúc của `_IMAGE_OPTIONAL_HEADER32`

- **ImageBase** là giá trị xác định địa chỉ bắt đầu của một file PE khi nó được nạp vào bộ nhớ, thường có giá trị là `0x400000`.
- **DataDirectory** là một mảng gồm có 16 phần tử `_IMAGE_DATA_DIRECTORY` chiếm 128 bytes và cũng là thành phần cuối cùng trong cấu trúc này, mỗi phần tử là một cấu trúc dữ liệu quan trọng trong file PE như Import Address Table (IAT), Import Directory, Export Directory, ...

Cấu trúc của một `_IMAGE_DATA_DIRECTORY` gồm hai phần và chiếm 8 bytes dữ liệu được thể hiện như sau.

```
1  typedef struct _IMAGE_DATA_DIRECTORY {
2      DWORD          VirtualAddress;
```

```

3     DWORD    Size;
4 } IMAGE_DATA_DIRECTORY, *PIMAGE_DATA_DIRECTORY;

```

Code 2.13: Cấu trúc của `_IMAGE_DATA_DIRECTORY`

- **VirtualAddress** là một địa chỉ ảo tương đối (Relative Virtual Address gọi tắt là RVA). Địa chỉ này được ánh xạ từ file vào trong bộ nhớ máy tính bằng cách lấy địa ImageBase cộng với địa chỉ thực. Ví dụ: nếu một file PE có một section nằm tại địa chỉ `0x1000` và được nạp tại địa chỉ `0x400000` (ImageBase) thì section sẽ có địa chỉ là `0x401000` trong vùng nhớ.
- **Size** là kích thước của cấu trúc dữ liệu.

Danh sách 16 phần tử trong mảng `DataDirectory` được thể hiện như sau.

| Index | Description                          |
|-------|--------------------------------------|
| 0     | IMAGE_DIRECTORY_ENTRY_EXPORT         |
| 1     | IMAGE_DIRECTORY_ENTRY_IMPORT         |
| 2     | IMAGE_DIRECTORY_ENTRY_RESOURCE       |
| 3     | IMAGE_DIRECTORY_ENTRY_EXCEPTION      |
| 4     | IMAGE_DIRECTORY_ENTRY_SECURITY       |
| 5     | IMAGE_DIRECTORY_ENTRY_BASERELOC      |
| 6     | IMAGE_DIRECTORY_ENTRY_DEBUG          |
| 7     | IMAGE_DIRECTORY_ENTRY_COPYRIGHT      |
| 8     | IMAGE_DIRECTORY_ENTRY_GLOBALPTR      |
| 9     | IMAGE_DIRECTORY_ENTRY_TLS            |
| 10    | IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG    |
| 11    | IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT   |
| 12    | IMAGE_DIRECTORY_ENTRY_IAT            |
| 13    | IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT   |
| 14    | IMAGE_DIRECTORY_ENTRY_COM_DESCRIPTOR |
| 15    | IMAGE_DIRECTORY_ENTRY_ENTRIES        |

Code 2.14: Các phần tử của mảng `DataDirectory`

### 2.4.5 Section Table

**Section Table** là thành phần tiếp theo ngay sau NT Headers. Vùng này chứa một danh sách các cấu trúc **IMAGE\_SECTION\_HEADER**, mỗi phần tử sẽ chứa thông tin về một section trong PE file. Section này chứa dữ liệu cụ thể như mã máy (Machine code), dữ liệu được khởi tạo và các loại dữ liệu khác. Mỗi một section sẽ đóng một vai trò cụ thể, chính điều này giúp cho việc tổ chức và quản lý dữ liệu hiệu quả hơn. Một vài section sẽ có tên đặc biệt dựa trên mục đích được sử dụng. Tất cả các tên section được miêu tả chi tiết tại trang Microsoft<sup>10</sup>. Tuy nhiên, một số chương trình sẽ có thêm những sections khác để phù hợp với những yêu cầu riêng biệt của chúng. Dưới đây chúng tôi chỉ liệt kê một số tên phổ biến được sử dụng:

- **Executable code Section (.text)** là phân vùng chứa mã thực thi.

<sup>10</sup><https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>

- **Read-only Data Section (.rdata)** là phân vùng chứa dữ liệu được khởi tạo và chỉ cho phép đọc (read-only).
- **.bss** là phân vùng chứa dữ liệu chưa được khởi tạo.
- **Resources Section (.rsrc)** là phân vùng chứa tài nguyên (ảnh, logo, ...).
- **Import Data Section (.idata)** là phân vùng chứa thông tin về tất cả các hàm được sử dụng bởi file PE từ các file DLLs.
- **Export Data Section (.edata)** là phân vùng chứa địa chỉ tới các hàm được sử dụng bởi một chương trình khác.

Một cấu trúc của **IMAGE\_SECTION\_HEADER** được định nghĩa như sau.

```

1  typedef struct _IMAGE_SECTION_HEADER {
2      BYTE      Name[IMAGE_SIZEOF_SHORT_NAME];
3      union {
4          DWORD   PhysicalAddress;
5          DWORD   VirtualSize;
6      } Misc;
7      DWORD      VirtualAddress;
8      DWORD      SizeOfRawData;
9      DWORD      PointerToRawData;
10     DWORD      PointerToRelocations;
11     DWORD      PointerToLinenumbers;
12     WORD       NumberOfRelocations;
13     WORD       NumberOfLinenumbers;
14     DWORD      Characteristics;
15 } IMAGE_SECTION_HEADER, *PIMAGE_SECTION_HEADER;

```

Code 2.15: Cấu trúc của **\_IMAGE\_SECTION\_HEADERS**

## 2.5 Kỹ thuật chèn mã (Hooking)

**Kỹ thuật chèn mã (Hooking)** là một phương pháp được áp dụng để theo dõi, sửa đổi hoặc chặn các lời gọi hàm từ các ứng dụng hoặc hệ thống. Trong quá trình này, một hàm được gọi để chặn hoặc giám sát một hàm khác, thường là các hàm API trong môi trường Windows, hoặc các sự kiện quan trọng trong hệ thống. Các hàm được dùng để chặn các hàm khác, được gọi là thủ tục Hook, có khả năng xử lý và ảnh hưởng lên mỗi sự kiện mà nó nhận được, bằng cách sửa đổi hoặc thậm chí loại bỏ sự kiện đó.

Sự linh hoạt của kỹ thuật Hooking cho phép các nhà phân tích mã độc hoặc nhà phát triển phần mềm có thể giám sát và kiểm soát các hoạt động của chương trình hoặc hệ thống một cách chi tiết và linh hoạt hơn. Nhờ vào việc theo dõi hành vi của chương trình, họ có thể đánh giá sự hoạt động của hệ thống, phát hiện các hoạt động không mong muốn, và thậm chí can thiệp vào hoạt động của chương trình nếu cần thiết.

Một số kỹ thuật hook được sử dụng phổ biến:



- 
- **Import Address Table (IAT) Hooking** là một kỹ thuật làm thay đổi các giá trị trong bảng IAT, một cấu trúc dữ liệu chứa con trỏ hàm trỏ tới địa chỉ của các hàm mà chương trình sẽ sử dụng từ các file thư viện (DLLs) trong quá trình thực thi. Bằng cách thay đổi giá trị con trỏ hàm trong IAT để trỏ đến một hàm khác, từ đó ta có thể theo dõi hoặc can thiệp vào hàm đó.
  - **Inline Hooking** kỹ thuật này thực hiện bằng cách thay đổi một số lệnh mã máy (machine code) của một hàm tại thời điểm chương trình thực thi. Chèn một số lệnh thay thế vào đầu hàm để chuyển luồng thực thi đến một hàm khác và sau đó có thể quay trở lại hàm gốc từ trong hàm Hooking nếu cần.

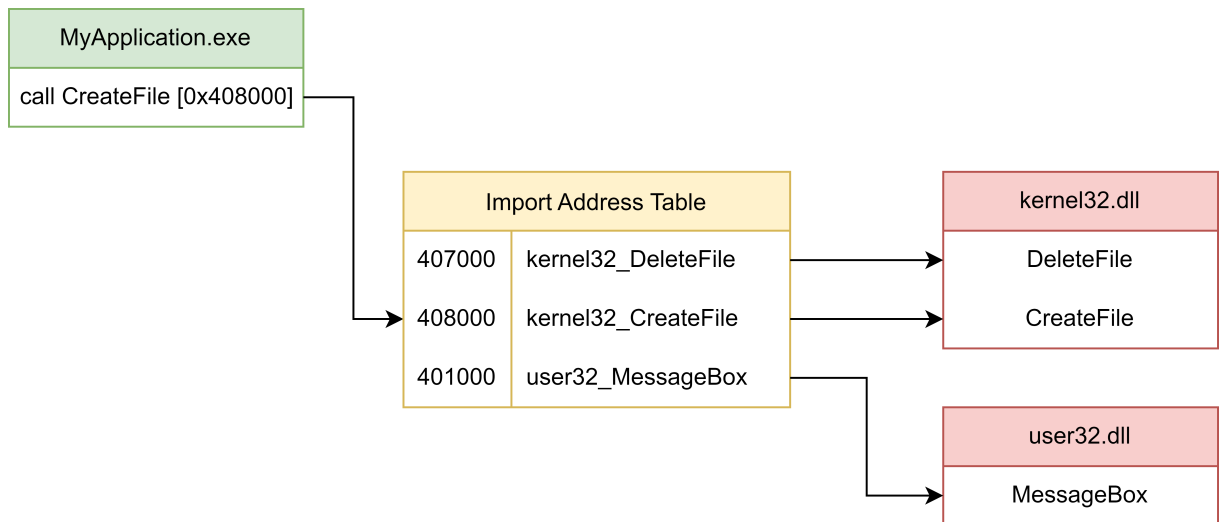
### 2.5.1 IAT Hooking

**Import Address Table (IAT)** là một thành phần quan trọng được định nghĩa trong cấu trúc của một file thực thi Portable Executable (PE). Được đặt trong phần Data Directory của PE header, IAT là một bảng dữ liệu chứa các con trỏ hàm trỏ tới các hàm được định nghĩa trong các file thư viện động (DLLs) khi chúng được nạp vào bộ nhớ.

Khi một chương trình thực thi được khởi động, trình tải (loader) của hệ điều hành sẽ thực hiện quá trình nạp các DLLs cần thiết vào bộ nhớ. Trong quá trình này, loader sẽ tạo ra IAT và điền địa chỉ của các hàm được định nghĩa trong các DLLs vào bảng này. Điều này cho phép chương trình thực thi truy cập vào các hàm trong các DLLs thông qua các con trỏ được lưu trữ trong IAT.

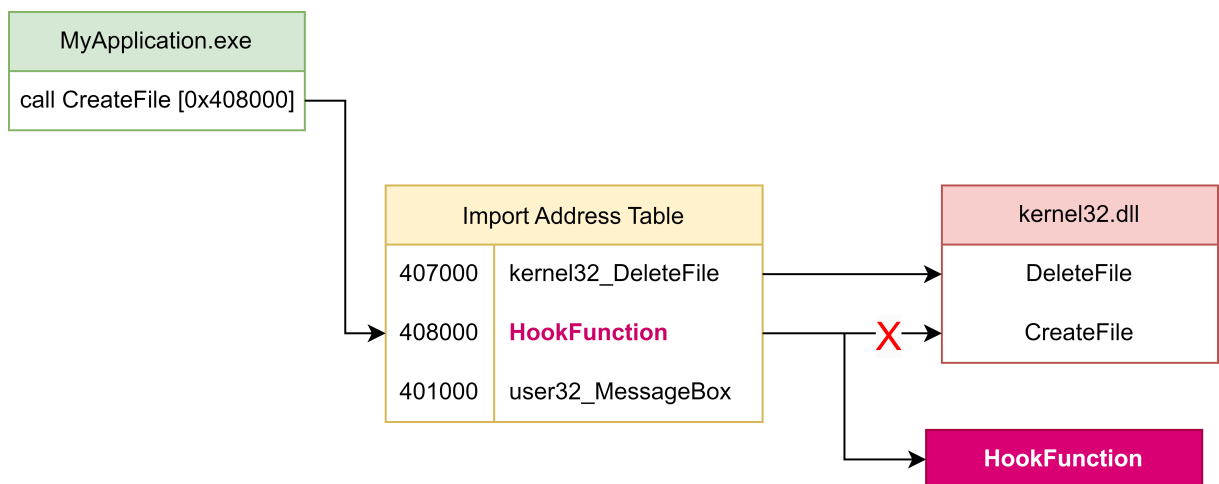
IAT là một phần cơ bản của quá trình liên kết động và đóng một vai trò quan trọng trong việc đảm bảo rằng chương trình thực thi có thể tương tác với các hàm trong các DLLs một cách hiệu quả và linh hoạt.

Hình 2.4 mô tả về cách hoạt động của IAT trong quá trình thực thi của một chương trình. Ví dụ: khi một hàm API CreateFile được gọi, lúc này chương trình sẽ tìm kiếm trong bảng IAT địa chỉ thực của hàm này (nằm trong thư viện kernel32.dll), sau đó chương trình sẽ thực hiện lệnh nhảy không điều kiện vào hàm trong thư viện và bắt đầu thực thi nó.



Hình 2.4: Cơ chế hoạt động của IAT

Bằng cách thay đổi các giá trị con trỏ hàm ở bảng IAT, ta có thể điều hướng luồng thực thi của chương trình sang một hàm mục tiêu cụ thể (hình 2.5). Quá trình này được diễn ra trong lúc chương trình thực thi.



Hình 2.5: Cơ chế hoạt động của IAT Hooking

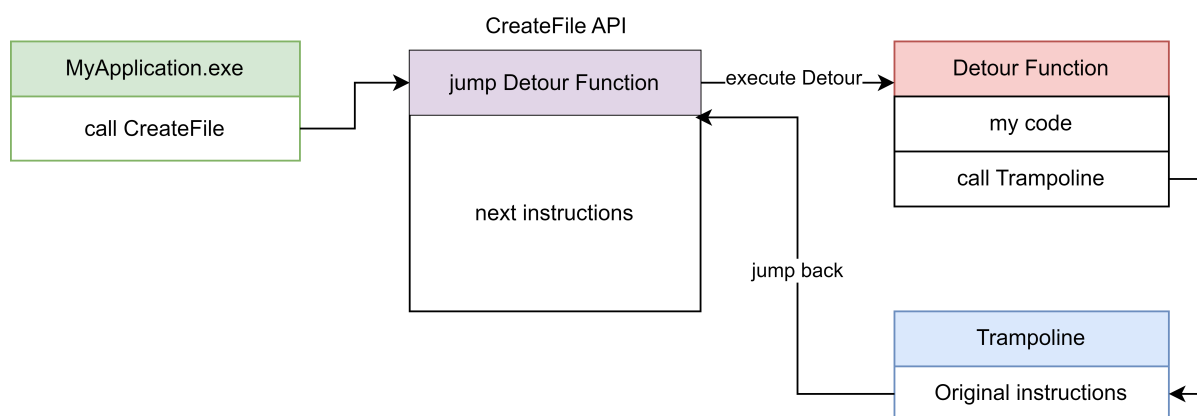
## 2.5.2 Inline Hooking

Khác với việc sửa đổi trực tiếp các con trỏ hàm trong bảng Import Address Table (IAT), đòi hỏi ghi đè một số byte ở đầu của một hàm API trong thư viện DLL của chương trình. Hành động này được thực hiện để chèn một lệnh nhảy tới một hàm mục tiêu cụ thể, mà thường được gọi là **Detours Function**. Cách tiếp cận này thường sử dụng các lệnh jmp, push/ret để điều hướng luồng thực thi. Trong trường hợp cần tái sử dụng hàm API đã bị ghi đè, kỹ thuật **Trampoline** được áp dụng. Kỹ thuật này bằng cách ghi đè lại các byte đã thay đổi trước đó. Chi tiết được mô tả ở hình 2.6.

Detours Function thường được chèn vào để thực hiện một số hành động cần

thiết trước hoặc sau khi gọi hàm gốc. Nó cũng có thể được sử dụng để gọi lại hàm gốc ban đầu thông qua Trampoline nếu cần. Trong khi Detours Function thực hiện các hành động được đặt ra, Trampoline được sử dụng để lưu trữ một số lệnh đã bị thay đổi của hàm bị Hook. Sau đó, Trampoline thực hiện lệnh nhảy tới lệnh tiếp theo trong hàm gốc, giúp bảo toàn các thay đổi và đảm bảo rằng hàm gốc vẫn được gọi khi cần thiết.

Kết quả là, phương pháp này tạo ra một cách tiếp cận linh hoạt và mạnh mẽ để thực hiện Hooking trên các hàm API, cho phép kiểm soát luồng thực thi, quan sát và thay đổi các tham số cũng như giá trị trả về của hàm và có thể thực hiện các hành động tùy chỉnh một cách hiệu quả.



Hình 2.6: Cơ chế hoạt động của Inline Hooking

## 2.6 Môi trường ảo hóa (Sandbox)

**Sandbox**[7] là một môi trường ảo hóa đặc biệt được thiết kế để chạy các ứng dụng, tập tin hoặc mã nguồn không đáng tin cậy mà không gây nguy hiểm cho hệ thống chính và các thiết bị mạng. Mục tiêu chính của Sandbox là cung cấp một không gian an toàn, nơi các mối đe dọa từ các nguồn không đáng tin cậy có thể được kiểm soát và ngăn chặn.

Có hai cách tiếp cận chính để đạt được mục tiêu này. Đầu tiên, là thông qua việc chặn một số hoạt động có khả năng gây nguy hiểm đến hệ thống bằng cách cô lập một phần tác vụ. Một cách tiếp cận khác là sử dụng môi trường ảo, nơi mà các tiến trình (processor), bộ nhớ (memory) và hệ thống file (file system) được giả lập. Môi trường ảo này được tổ chức và quản lý một cách nghiêm ngặt, tạo ra một biên giới an toàn giữa các ứng dụng không đáng tin cậy và hệ thống.

Trong một ngữ cảnh rộng lớn hơn, Sandbox cung cấp một cơ chế quan trọng cho các nhà nghiên cứu và chuyên gia an toàn thông tin để phân tích, phát hiện và ngăn chặn các mối đe dọa tiềm ẩn. Bằng cách tạo ra một môi trường kiểm soát và an toàn, Sandbox cho phép các chuyên gia theo dõi và nghiên cứu các hành vi độc hại của các ứng dụng và tập tin, từ đó phát triển các biện pháp phòng ngừa hiệu quả và bảo vệ hệ thống khỏi các cuộc tấn công tiềm ẩn.

Sandbox được sử dụng để phân tích các hành vi của một mã độc bằng cách

---

ghi nhận các hàm API được sử dụng hoặc phân tích vùng nhớ của hệ thống. Dưới đây là một số tác vụ, hành vi thường được sử dụng cho việc phân tích mã độc, và thường được ghi nhận trong các báo cáo mã độc:

- Giám sát hoạt động của các tiến trình (Processes) và luồng (Threads).
- Giám sát cơ sở dữ liệu của hệ thống Windows (Registry).
- Giám sát sự thay đổi của hệ thống tệp tin và thư mục.
- Giám sát hệ thống mạng.
- Giám sát các vùng nhớ của hệ thống.

## Các công cụ và nghiên cứu liên quan

### 3.1 Các công cụ

#### 3.1.1 Frida

**Frida**<sup>1</sup> là công cụ cho phép đo lường ứng dụng trong lúc thực thi (dynamic instrumentation tool), không chỉ là một công cụ mạnh mẽ, mà còn là một công cụ độc đáo và linh hoạt trong việc can thiệp và tương tác với ứng dụng đang chạy trên nhiều nền tảng khác nhau như: Windows, MacOS, Android, iOS.

Một trong những tính năng ấn tượng của Frida là khả năng lấy thông tin về tham số của các hàm và giá trị trả về của chúng trong quá trình thực thi. Điều này cho phép người dùng theo dõi và hiểu rõ hơn về cách hoạt động của các hàm trong ứng dụng, giúp tăng cường quá trình phân tích và debug.

Hơn nữa, Frida cũng cung cấp cho người dùng khả năng thay đổi hành vi của các hàm một cách linh hoạt và dễ dàng. Bằng cách chèn các đoạn mã hook vào trong quá trình thực thi của ứng dụng, người dùng có thể can thiệp vào các hàm và thay đổi chúng theo ý muốn. Điều này mở ra một loạt các ứng dụng sử dụng, từ việc vượt qua các cơ chế bảo vệ đến việc thực hiện các thử nghiệm và kiểm tra hệ thống.

Dưới đây là một số tính năng mạnh mẽ của Frida:

- **Truy cập và can thiệp vào hàm:** Frida cho phép gọi các hàm trong mã máy của ứng dụng và can thiệp vào chúng trực tiếp từ các ngôn ngữ cao cấp như Python hoặc JavaScript.
- **Hooking và Monitoring:** Bằng cách sử dụng Frida, bạn có thể hook vào các hàm và phương thức của ứng dụng để theo dõi và điều khiển hành vi của chúng trong quá trình thực thi.
- **Hỗ trợ trên nhiều nền tảng khác nhau:** Frida hỗ trợ nhiều nền tảng như Android, iOS, Windows, macOS và Linux, giúp phân tích ứng dụng trên nhiều hệ điều hành khác nhau.

---

<sup>1</sup><https://frida.re/>

- 
- **Dynamic Instrumentation** (chèn mã trong lúc chương trình thực thi): Thay vì phải sửa đổi mã nguồn của ứng dụng và biên dịch lại, Frida sử dụng kỹ thuật chèn mã trong lúc thực thi (Dynamic Instrumentation) để can thiệp vào ứng dụng và có thể thay đổi chức năng của chương trình.

Tóm lại, Frida không chỉ là một công cụ mạnh mẽ để can thiệp và tương tác với ứng dụng đang chạy, mà còn là một công cụ linh hoạt và dễ dàng sử dụng trên nhiều nền tảng khác nhau. Sự tích hợp và ứng dụng rộng rãi của Frida đã đem lại nhiều giá trị và tiện ích cho cả các nhà phát triển phần mềm và các chuyên gia bảo mật mạng.

### 3.1.2 Yara

**Yara**<sup>2</sup> là một công cụ mạnh mẽ và linh hoạt được sử dụng rộng rãi trong lĩnh vực bảo mật thông tin, được phát triển bởi **VirusTotal**<sup>3</sup> - một trong những công ty hàng đầu trong việc phân tích mã độc và đánh giá rủi ro. Công cụ này là một tập hợp các quy tắc được người sử dụng tự viết để kiểm tra và phát hiện các chuỗi kí tự hoặc các chuỗi byte cụ thể trong một file hoặc vùng nhớ của tiến trình.

Với Yara, người sử dụng có thể tạo ra các quy tắc gọi là **Yara Rules**, dựa trên các đặc điểm nổi bật nhất của mã độc, bao gồm cả các chuỗi kí tự, chuỗi byte, hoặc các đặc điểm cấu trúc đặc biệt. Các quy tắc này sau đó được áp dụng vào các file hoặc vùng nhớ của tiến trình để kiểm tra xem chúng có chứa các đặc điểm tương tự với các mẫu mã độc đã biết hay không.

Một trong những điểm mạnh của Yara là tính linh hoạt của nó. Người sử dụng có thể tạo ra các quy tắc phong phú và phức tạp để phát hiện nhiều loại mã độc khác nhau, từ các mối đe dọa phổ biến đến các biến thể mới và không rõ nguồn gốc. Hơn nữa, Yara cũng hỗ trợ sử dụng biểu thức chính quy và các toán tử logic để xây dựng các quy tắc mạnh mẽ và chính xác.

Công cụ này không chỉ hữu ích cho các chuyên gia bảo mật thông tin và nhóm phòng chống mã độc, mà còn có thể được tích hợp vào các hệ thống tự động hóa và quy trình phân tích mã độc tự động. Điều này giúp cải thiện hiệu suất và tính hiệu quả trong việc phát hiện và phản ứng với các mối đe dọa mạng và mã độc mới nhanh chóng.

Một số thông tin cơ bản của Yara Rules bao gồm các thành phần sau:

- **meta** (thông tin tiêu đề): chứa thông tin mô tả thuộc tính cho một quy tắc. Các thuộc tính này có thể là thông tin cơ bản để mô tả về quy tắc (description), tác giả (author), phiên bản (version), hoặc bất kỳ thông tin nào khác mà người viết quy tắc muốn.
- **string** (chuỗi các quy tắc): dùng để xác định mẫu của dữ liệu muốn tìm kiếm trong file hoặc vùng nhớ. Các mẫu này có thể biểu diễn dưới dạng chuỗi kí tự, các byte liên tiếp nhau.

---

<sup>2</sup><https://github.com/VirusTotal/yara>

<sup>3</sup><https://www.virustotal.com/>

- **condition** (điều kiện để khớp với quy tắc): xác định các điều kiện mà một file hoặc vùng nhớ phải thỏa mãn để phù hợp với quy tắc đó.

Ví dụ về một quy tắc giúp phát hiện file thực thi **DotNET**:

```
1 import "pe"
2 rule is_net_exe
3 {
4     meta:
5         description = "DotNet Executable File"
6     condition:
7         pe.imports ("mscorlib.dll", "_CorExeMain")
8 }
```

Code 3.1: Nhận dạng file thực thi DotNet

Quy tắc trên kiểm tra một file thực thi có phải là thuộc loại DotNet hay không. Bằng cách kiểm tra có tồn tại hàm **\_CorExeMain** trong thư viện **mscorlib.dll** hay không. Đây là đặc điểm nổi bật chỉ có trong file thực thi DotNet.

## 3.2 Một số nghiên cứu liên quan

### 3.2.1 Nhận dạng Fileless PowerShell Malware

Việc nhận dạng và phát hiện được Fileless PowerShell Malware là một điều hết sức khó khăn do các tính chất cũng như hành vi không rõ ràng. Bởi PowerShell là ngôn ngữ script nên đoạn thực thi PowerShell có thể bị biến đổi bằng nhiều hình thức khác nhau, và nhanh chóng triển khai mà không cần phải “compile” thành file thực thi, khiến cho việc triển khai mã độc theo dạng PowerShell dễ dàng qua mặt các khả năng nhận diện theo hash, hay Signature. Bên cạnh đó, sử dụng các cơ chế bảo vệ giúp việc phân tích tĩnh gặp rất nhiều khó khăn. Các hành vi của nó chỉ được thể hiện rõ trong quá trình thực thi. Vì vậy cần có một công cụ có thể thực thi loại mã độc này và giám sát các hành vi của nó. Một số cách có thể hiện thực như: Sandbox, Execution Emulation[15].

- **Sandbox** là một trong những phương pháp hiện thực đáng chú ý nhất. Khi mã độc được thực thi trong một môi trường Sandbox, mọi hoạt động của nó đều được giám sát cẩn thận. Các hàm API có khả năng gây hại tới hệ thống thường bị ngăn chặn, trong khi các hành vi độc hại khác được theo dõi và ghi lại. Các kỹ thuật hooking có thể được sử dụng để can thiệp vào quá trình thực thi của mã độc và kiểm soát các hàm API, tăng cường khả năng ngăn chặn và phát hiện các hành vi độc hại.
- **Execution Emulation** là một phương pháp hiệu quả để nhận dạng và phát hiện mã độc Fileless PowerShell. Bằng cách giả lập môi trường thực thi của một đoạn mã PowerShell, người dùng có thể theo dõi hành vi của mã độc mà không cần thực sự chạy nó trên hệ thống thật. Công cụ giả lập mã nguồn mở

---

như **Qemu**<sup>4</sup> và **Qiling**<sup>5</sup> là những công cụ mạnh mẽ và linh hoạt có thể được sử dụng để thực hiện việc này.

Tiếp theo, chúng tôi sẽ trình bày một số công cụ hỗ trợ phân tích mã độc dựa vào hai kỹ thuật chính là phân tích tĩnh và phân tích động (dùng Sandex). Với kỹ thuật phân tích tĩnh, chúng tôi sẽ giới thiệu hai công cụ là: Microsoft Copilot và AMSI. Còn với kỹ thuật phân tích động, chúng tôi sẽ giới thiệu AnyRun.

### 3.2.1.1 Microsoft Copilot

**Microsoft Copilot**<sup>6</sup> là một công cụ trí tuệ nhân tạo đột phá, được phát triển bởi Microsoft, nhằm hỗ trợ người dùng trong nhiều tình huống khác nhau. Tính năng tiêu biểu của Copilot là khả năng cung cấp gợi ý và hỗ trợ trong việc viết code, từ các đoạn mã ngắn đến các hàm phức tạp, giúp tăng cường hiệu suất và tiện ích cho các nhà phát triển.

Ngoài việc hỗ trợ viết chương trình, Copilot còn có thể được sử dụng để giải đáp các câu hỏi và tìm kiếm thông tin. Dựa trên một cơ sở dữ liệu lớn về kiến thức và thông tin, Copilot có thể cung cấp câu trả lời và thông tin chi tiết về nhiều chủ đề khác nhau, từ khoa học máy tính đến văn hóa và nghệ thuật.

Trong lĩnh vực bảo mật thông tin, công cụ này cũng có thể giúp phân tích các mã độc, bao gồm cả các đoạn mã PowerShell. Với khả năng hiểu và tự động hoàn thiện code, Copilot có thể giúp người dùng phân tích cấu trúc và chức năng của mã độc PowerShell một cách chi tiết và tin cậy. Tuy nhiên, có một số hạn chế khi phân tích các mã độc PowerShell đã được Obfuscation. Các kỹ thuật Obfuscation như mã hóa, tạo ra mã rối và giấu đi các thông tin quan trọng có thể làm cho quá trình phân tích trở nên khó khăn hơn, và Copilot có thể gặp khó khăn trong việc hiểu rõ cấu trúc và chức năng của các mã trong trường hợp này.

Tóm lại, Microsoft Copilot là một công cụ mạnh mẽ và đa dạng, có thể hỗ trợ người dùng trong nhiều khía cạnh của việc phát triển phần mềm và bảo mật thông tin. Tuy nhiên, việc sử dụng Copilot cần phải cân nhắc kỹ lưỡng và kết hợp với các công cụ và kỹ thuật khác để đảm bảo tính chính xác và độ tin cậy của kết quả.

### 3.2.1.2 Antimalware Scan Interface

**Antimalware Scan Interface**<sup>7</sup> (AMSI) là một công cụ bảo mật do Microsoft phát triển và được tích hợp vào trong hệ điều hành Windows. Công cụ này giúp phát hiện các mối đe dọa từ mã độc ngay từ khi nó tồn tại trên hệ thống và ngăn chặn nó để bảo vệ hệ thống. AMSI hoạt động bằng cách tìm kiếm các Signature từ các mối đe dọa và kiểm tra nó trong cơ sở dữ liệu đã có sẵn. Nếu như phát hiện có tồn tại thì hệ thống sẽ cảnh báo đến người dùng và tự động loại bỏ nó ra khỏi hệ thống. Tuy nhiên, công cụ này vẫn còn điểm yếu trước các cuộc tấn công bằng các ngôn ngữ scripting như: PowerShell, JavaScript, VBScript. Bằng cách sử dụng kỹ

---

<sup>4</sup><https://www.qemu.org/>

<sup>5</sup><https://qiling.io/>

<sup>6</sup><https://copilot.microsoft.com/>

<sup>7</sup><https://learn.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal>



---

thuật Obfuscation (đã được đề cập ở tiểu mục 2.2.1) thì lúc này hệ thống AMSI sẽ không thể nhận diện được bằng Signature được nữa.

Ngoài kỹ thuật Obfuscation, kẻ tấn công còn sử dụng nhiều phương pháp tinh vi khác để vượt qua cơ chế bảo vệ của AMSI. Một trong những kỹ thuật phổ biến là Hooking, được trình bày chi tiết ở mục 2.5. Hooking cho phép tin tặc can thiệp và thay đổi hành vi của các hàm API hệ thống, từ đó vô hiệu hóa hoặc qua mặt các biện pháp kiểm tra của AMSI.

Không chỉ dừng lại ở đó, cộng đồng mạng cũng chia sẻ rộng rãi các kỹ thuật để vượt qua hệ thống AMSI, chẳng hạn như **Amsi-Bypass-Powershell**<sup>8</sup>, cung cấp nhiều phương pháp và đoạn mã giúp thực hiện việc này. Ngoài ra, còn có các công cụ trực tuyến như AMSI.fail<sup>9</sup>, cho phép người dùng tạo tự động các đoạn script PowerShell đã bị làm rối mã. Các công cụ này giúp mã độc dễ dàng vượt qua các cơ chế bảo vệ của AMSI mà không bị phát hiện.

### 3.2.1.3 AnyRun

**AnyRun**<sup>10</sup> là một nền tảng phân tích mã độc, cung cấp một giao diện trực quan cho phép thực hiện và quan sát các hoạt động của các mã độc trong một môi trường ảo hóa. Người dùng có thể tương tác trực tiếp với các phần mềm độc hại trong khi chúng chạy và có thể thực hiện các hành động để xem phản ứng của các phần mềm. Sau khi hoàn thành phân tích, AnyRun cung cấp các báo cáo chi tiết về các hành vi của mã độc, bao gồm tác động của chương trình vào hệ thống file, kết nối mạng, tạo process hay thread, và các thay đổi trong registry.

AnyRun hỗ trợ nhiều phiên bản hệ điều hành Windows, giúp người dùng có thể phân tích phần mềm độc hại trên các môi trường khác nhau và đảm bảo rằng các phân tích của người dùng phù hợp với thực tế.

## 3.2.2 Kỹ thuật chống máy ảo (Anti Virtual Machine)

Việc sử dụng máy ảo (Virtual Machines) và các công cụ Sandbox đã trở thành một phương pháp phổ biến và hiệu quả để kiểm tra các tệp tin và hành vi đáng ngờ. Những môi trường ảo này cho phép kiểm tra các mối đe dọa từ mã độc một cách an toàn mà không gây nguy hiểm cho hệ thống thực. Tuy nhiên, kẻ tấn công không ngừng tìm kiếm các cách thức mới để vượt qua các biện pháp phòng thủ này. Việc sử dụng máy ảo có thể đem tới các khác biệt so với sử dụng máy thật (máy nạn nhân), từ đó các kỹ thuật phát hiện máy ảo lúc đang chạy ra đời, gọi chung là **Kỹ thuật chống máy ảo** (Anti Virtual Machine).

Kỹ thuật này cho phép mã độc phát hiện ra rằng nó đang chạy trong môi trường ảo hoặc Sandbox. Nếu phát hiện được điều này, mã độc sẽ thay đổi hành vi hoặc ngừng hoạt động hoàn toàn, khiến việc phân tích và phát hiện trở nên khó khăn hơn rất nhiều. Kẻ tấn công sử dụng các đoạn script ngắn gọn và hiệu quả để thực hiện quá trình này. Những script này có thể kiểm tra các dấu hiệu cụ thể của

---

<sup>8</sup><https://github.com/S3cur3Th1sSh1t/Amsi-Bypass-Powershell>

<sup>9</sup><https://amsi.fail/>

<sup>10</sup><https://any.run/>

---

môi trường ảo như tên của các tệp tin hệ thống đặc thù, các dịch vụ chạy ngầm, cấu hình phần cứng giả lập, và nhiều yếu tố khác mà chỉ có trong máy ảo hoặc Sandbox. Từ đó, có thể đánh lừa người dùng, khiến cho họ nghĩ đây là một file vô hại và mở nó trên hệ thống thật.

```
PS C:\Users\LAP12512-local> (Get-WmiObject win32_computersystem).model  
VMware20,1  
PS C:\Users\LAP12512-local>
```

Hình 3.1: Kiểm tra hệ thống Virtual Machine

Kỹ thuật này còn có thể kết hợp với các phương pháp khác như Obfuscation để tăng cường khả năng ẩn mình. Điều này tạo ra một lớp bảo vệ đa chiều, khiến cho việc phân tích và phát hiện mã độc trở nên phức tạp và tốn thời gian hơn.

Ví dụ một đoạn command ngắn gọn dưới đây, chúng ta có thể biết được hệ thống đang chạy trên máy ảo VMware (Hình 3.1).

## Đề xuất giải pháp

Trong luận văn này, công cụ được hiện thực có thể ghi nhận được các hành vi của một mã độc Ransomware. Các hành vi của mã độc này đã được miêu tả chi tiết ở tiểu mục 2.1.2. Công cụ có thể chạy trên hệ điều hành Windows 10 và 11.

### 4.1 Đề xuất phương pháp

Do mã độc sử dụng các kỹ thuật Obfuscation và sự hạn chế của kỹ thuật phát hiện bằng Signature không còn hiệu quả. Do đó cần phải thực hiện một công cụ cho phép thực thi và giám sát các hành vi của nó. Từ đó có thể nhận diện được đây có phải là chương trình có hành vi độc hại hay không. Các phương pháp nhận diện đã được chúng tôi đề cập ở tiểu mục 3.2.1. Mỗi một phương pháp đều có một ưu điểm và nhược điểm riêng.

Với kỹ thuật Execution Emulation, sẽ sử dụng một số công cụ như qiling, qemu, . . . để có thể giả lập lại quá trình thực thi của mã độc mà không cần phải trực tiếp thực thi nó. Quá trình giả lập chương trình này sẽ hoàn toàn không gây ảnh hưởng tới hệ thống. Tuy nhiên, cách tiếp cận này thì phải cần tìm hiểu rất nhiều về cách mà một ngôn ngữ được biên dịch và chạy trên hệ thống. Điều này bao gồm việc nắm vững các khái niệm về kiến trúc hệ thống, quá trình biên dịch mã nguồn thành mã máy, và cách mã độc tương tác với hệ điều hành và phần cứng.

Với kỹ thuật Sandbox, chúng tôi tiếp tục đối diện với hai bài toán nhỏ hơn. Thứ nhất, để thực hiện Sandbox chúng tôi có thể xây dựng một môi trường hoàn toàn cô lập, tương tự như một máy ảo. Cách tiếp cận thứ hai, chúng tôi chỉ tập trung chặn một số chức năng của mã độc có thể tác động tới hệ thống thật. Với cách tiếp cận đầu tiên, việc tạo ra một môi trường cô lập hoàn toàn như một máy ảo yêu cầu sự đầu tư lớn về thời gian và kiến thức chuyên sâu về hệ thống máy tính. Quá trình này đòi hỏi phải thiết lập và cấu hình các yếu tố ảo hóa, đảm bảo rằng mọi hành vi của mã độc đều được giới hạn trong môi trường an toàn, không thể gây hại cho hệ thống thật. Mặt khác, cách tiếp cận thứ hai tập trung vào việc ngăn chặn các chức năng nguy hiểm của mã độc. Bằng cách chặn và giám sát các lệnh và hoạt động mà mã độc có thể sử dụng để tấn công hệ thống, từ đó có thể giảm thiểu rủi ro mà không cần thiết lập một môi trường hoàn toàn cô lập.

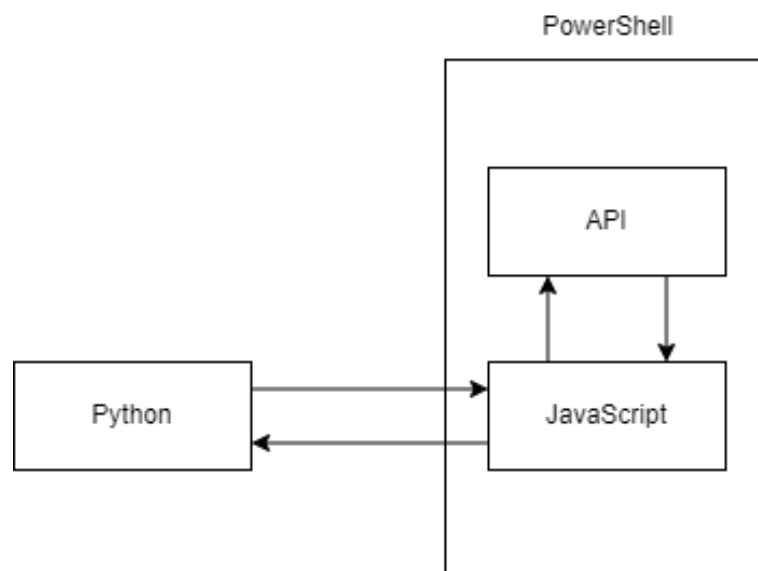
Qua một thời gian tìm hiểu sâu về các phương pháp này, chúng tôi nhận thấy rằng thực hiện Sandbox bằng cách chặn một số hành vi độc hại sẽ khả quan hơn. Bằng cách sử dụng kỹ thuật Hooking (đã nêu ở mục 2.5), sử dụng công cụ Frida (đã nêu ở mục 3.1.1) để hỗ trợ quá trình Hooking, bên cạnh đó chúng tôi sẽ sử dụng Yara (đã nêu ở mục 3.1.2) để hỗ trợ quá trình nhận diện nhanh các Signature của một số loại file phổ biến. Chính vì vậy, công cụ Sandbox của chúng tôi sẽ được thiết kế đơn giản và nhẹ hơn do không cần phải xây dựng lại một máy ảo, nhưng vẫn đảm bảo được tính cô lập của hệ thống.

Thông qua việc giám sát, kiểm tra, thay đổi được các Input và Output của một hàm API bằng kỹ thuật Hooking. Từ đó, công cụ Sandbox sẽ có thể biết được hành vi của một chương trình thông qua các hàm API được gọi. Để từ đó có thể đưa ra một kết luận tương đối chính xác về hành vi của chương trình PowerShell cần được kiểm tra có phải là Ransomware hoặc là một hành vi độc hại của một loại mã độc phân tán khác hay không.

## 4.2 Ngôn ngữ và các thư viện được sử dụng

Chúng tôi sử dụng hai ngôn ngữ để có thể hiện thực công cụ này.

- **JavaScript**<sup>1</sup>: dùng để viết các hàm hook vào các hàm API trong tiến trình PowerShell đang chạy.
- **Python**<sup>2</sup>: dùng để thực thi công cụ, tương tác với Frida, xử lý dữ liệu do JavaScript gửi về.



Hình 4.1: Tổng quan về ngôn ngữ được sử dụng trong hệ thống

<sup>1</sup><https://www.javascript.com/>

<sup>2</sup><https://www.python.org/>

---

Đối với ngôn ngữ Python thì sẽ cài đặt thêm một số thư viện bổ sung:

- **frida-tools** phiên bản 12.3.0.
- **yara-python** phiên bản 4.4.0.
- **requests** phiên bản tối thiểu 2.26.0.

## Phân tích các hàm API

Trong chương này, chúng tôi sẽ đi sâu vào việc phân tích các hàm API quan trọng được sử dụng để ngăn chặn các hành vi của mã độc. Chúng tôi sẽ không chỉ dừng lại ở việc liệt kê các hàm API, mà còn tập trung phân tích chức năng của từng hàm và các tham số quan trọng cần thiết cho quá trình giám sát.

### 5.1 Files

#### 5.1.1 CreateFile

Đây là hàm được dùng để tạo một file mới, đồng thời cũng được dùng để mở các file đã tồn tại trong hệ thống tùy thuộc vào giá trị tham số của hàm.

```

1  HANDLE CreateFile(
2      LPCSTR                lpFileName ,
3      DWORD                 dwDesiredAccess ,
4      DWORD                 dwShareMode ,
5      LPSECURITY_ATTRIBUTES lpSecurityAttributes ,
6      DWORD                 dwCreationDisposition ,
7      DWORD                 dwFlagsAndAttributes ,
8      HANDLE                hTemplateFile
9  );

```

Code 5.1: Cấu trúc của hàm CreateFile

Do hàm có rất nhiều tham số, tuy nhiên trong quá trình thực hiện thì chúng tôi chỉ lấy những thông tin cần thiết để xử lý. Một số thông tin về các tham số:

- **lpFileName** tên và vị trí của file trong hệ thống cần tạo hoặc mở.
- **dwCreationDisposition** tham số này giúp cho hệ thống biết cần tạo file hay mở file đã tồn tại. Có 4 giá trị: **CREATE\_ALWAYS**, **CREATE\_NEW**, **OPEN\_ALWAYS**, **OPEN\_EXISTING**.

### 5.1.2 WriteFile và ReadWrite

**WriteFile** và **ReadWrite** là hai hàm được dùng để đọc và ghi dữ liệu từ một file xác định. Hai hàm này có các tham số giống nhau nên chúng tôi chỉ tập trung trình bày hàm **WriteFile**:

```
1  BOOL WriteFile(  
2      HANDLE          hFile ,  
3      LPCVOID         lpBuffer ,  
4      DWORD           nNumberOfBytesToWrite ,  
5      LPDWORD         lpNumberOfBytesWritten ,  
6      LPOVERLAPPED    lpOverlapped  
7  )
```

Code 5.2: Cấu trúc của hàm NTWriteFile

- **hFile** giá trị xác định file nào sẽ được ghi.
- **lpBuffer** dùng để chứa dữ liệu được ghi vào file.
- **nNumberOfBytesToWrite** kích thước của dữ liệu được ghi vào file.

### 5.1.3 DeleteFile

Hàm dùng để xóa một file xác định trong hệ thống. Tham số truyền vào là đường dẫn tới một file cần xóa. Hàm sẽ trả về **True** nếu xóa thành công, ngược lại trả về **False**.

Đầu tiên, hàm kiểm tra xem tệp tin tồn tại hay không bằng cách thử mở nó để đọc. Nếu quá trình mở tệp tin không thành công, điều này ngụ ý rằng tệp tin không tồn tại hoặc không thể truy cập được. Trong trường hợp này, hàm sẽ trả về giá trị **False**.

Nếu file có thể mở được, hàm sẽ xóa file. Nếu quá trình xóa thành công, hàm sẽ trả về giá trị **True**, ngược lại sẽ trả về giá trị **False**.

```
1  BOOL DeleteFile(  
2      LPCTSTR lpFileName  
3  );
```

Code 5.3: Cấu trúc của hàm DeleteFile

### 5.1.4 MoveFile và CopyFile

**MoveFile** và **CopyFile** dùng để di chuyển hoặc sao chép một file đã tồn tại tới một file mới. Hàm trả về **True** nếu thực hiện thành công, ngược lại sẽ trả về **False**.

```
1  BOOL MoveFile(  
2      LPCTSTR lpExistingFileName ,  
3      LPCTSTR lpNewFileName  
4  )  
5  BOOL CopyFile(  
6      LPCTSTR lpExistingFileName ,  
7      LPCTSTR lpNewFileName ,  
8      BOOL     bFailIfExists
```

```

6      LPCSTR      lpExistingFileName ,
7      LPCSTR      lpNewFileName ,
8      BOOL        bFailIfExists
9  )

```

Code 5.4: Cấu trúc của hàm MoveFile và CopyFile

- **lpExistingFileName** đường dẫn tới file cần di chuyển hoặc sao chép.
- **lpNewFileName** đường dẫn tới file đích.

## 5.2 Registry

### 5.2.1 RegCreateKey và RegCreateKeyEx

Các hàm được dùng để tạo một khóa trong Registry.

```

1      LSTATUS RegCreateKey(
2          HKEY      hKey,
3          LPCSTR     lpSubKey,
4          PHKEY      phkResult
5      )
6      LSTATUS RegCreateKeyEx(
7          HKEY      hKey,
8          LPCSTR     lpSubKey,
9          DWORD      Reserved,
10         LPSTR      lpClass,
11         DWORD      dwOptions,
12         REGSAM     samDesired,
13         const LPSECURITY_ATTRIBUTES lpSecurityAttributes,
14         PHKEY      phkResult,
15         LPDWORD     lpdwDisposition
16     )

```

Code 5.5: Cấu trúc của hàm RegCreateKey và RegCreateKeyEx

- **hKey** giá trị Handle tới một khóa của Registry sẽ được tạo hoặc mở.
- **lpSubKey** tên của khóa con sẽ được tạo hoặc mở.
- **phkResult** lưu trữ Handle tới khóa được tạo hoặc mở.
- **lpdwDisposition** lưu trữ giá trị cho biết liệu khóa đó sẽ được tạo hay mở khóa đã tồn tại trước. Có hai giá trị là: **REG\_CREATED\_NEW\_KEY** hoặc **REG\_OPENED\_EXISTING\_KEY**.

### 5.2.2 RegOpenKey và RegOpenKeyEx

Các hàm được dùng để mở một khóa trong Registry. Các tham số trong hàm này tương tự với các hàm ở trên.



```

1  LSTATUS RegOpenKey (
2      HKEY          hKey ,
3      LPCWSTR       lpSubKey ,
4      PHKEY         phkResult
5  )
6  LSTATUS RegOpenKeyEx (
7      HKEY          hKey ,
8      LPCWSTR       lpSubKey ,
9      DWORD         ulOptions ,
10     REGSAM        samDesired ,
11     PHKEY         phkResult
12 )

```

Code 5.6: Cấu trúc của hàm RegOpenKey và RegOpenKeyEx

### 5.2.3 RegSetValue và RegSetValueEx

Các hàm được dùng để gán giá trị cho một khóa trong Registry.

```

1  LSTATUS RegSetValue (
2      HKEY          hKey ,
3      LPCWSTR       lpSubKey ,
4      DWORD         dwType ,
5      LPCSTR        lpData ,
6      DWORD         cbData
7  )
8  LSTATUS RegSetValueEx (
9      HKEY          hKey ,
10     LPCWSTR       lpValueName ,
11     DWORD         Reserved ,
12     DWORD         dwType ,
13     const BYTE     *lpData ,
14     DWORD         cbData
15 )

```

Code 5.7: Cấu trúc của hàm RegSetValue và RegSetValueEx

- **lpData** chứa dữ liệu sẽ được ghi vào khóa.
- **cbData** kích thước của dữ liệu được ghi.

### 5.2.4 RegDeleteKey và RegDeleteKeyEx

Các hàm được dùng để xóa một khóa trong Registry.

```

1  LSTATUS RegDeleteKey (
2      HKEY          hKey ,
3      LPCWSTR       lpSubKey
4  )
5  LSTATUS RegDeleteKeyEx (
6      HKEY          hKey ,
7      LPCWSTR       lpSubKey ,
8      REGSAM        samDesired ,

```

```

9         DWORD           Reserved
10     )

```

Code 5.8: Cấu trúc của hàm RegDeleteKey và RegDeleteKeyEx

- **lpSubKey** tên của khóa cần được xóa.

### 5.2.5 RegDeleteValue và RegDeleteKeyValue

```

1     LSTATUS RegDeleteValue(
2         HKEY           hKey,
3         LPCWSTR        lpValueName
4     )
5     LSTATUS RegDeleteKeyValue(
6         HKEY           hKey,
7         LPCWSTR        lpSubKey,
8         LPCWSTR        lpValueName
9     )

```

Code 5.9: Cấu trúc của hàm RegDeleteValue và RegDeleteKeyValue

- **lpSubKey** tên của khóa có giá trị cần được xóa.
- **lpValueName** tên giá trị cần xóa.

## 5.3 Process

### 5.3.1 CreateProcess

**CreateProcess** được dùng để tạo một Process mới thực thi trên hệ thống.

```

1     BOOL CreateProcess(
2         LPCSTR           lpApplicationName,
3         LPSTR            lpCommandLine,
4         LPSECURITY_ATTRIBUTES lpProcessAttributes,
5         LPSECURITY_ATTRIBUTES lpThreadAttributes,
6         BOOL             bInheritHandles,
7         DWORD            dwCreationFlags,
8         LPVOID           lpEnvironment,
9         LPCSTR           lpCurrentDirectory,
10        LPSTARTUPINFOA    lpStartupInfo,
11        LPPROCESS_INFORMATION lpProcessInformation
12    )

```

Code 5.10: Cấu trúc của hàm CreateProcess

- **lpApplicationName** tên của ứng dụng cần tạo.
- **lpCommandLine** tham số được truyền vào cho ứng dụng.
- **lpProcessInformation** thông tin về Process được tạo. Bao gồm Handle và giá trị PID của Process và Thread.

---

## 5.3.2 OpenProcess

**OpenProcess** được dùng để lấy giá trị Handle của Process thông qua giá trị PID.

```
1  HANDLE OpenProcess(  
2      DWORD          dwDesiredAccess ,  
3      BOOL           bInheritHandle ,  
4      DWORD          dwProcessId  
5  )
```

Code 5.11: Cấu trúc của hàm OpenProcess

- **dwProcessId** giá trị PID của Process cần được mở.

## 5.4 Internet

### 5.4.1 GetAddrInfo

**GetAddrInfo** được sử dụng để lấy thông tin về địa chỉ IP và cổng của một máy chủ dựa trên tên miền hoặc dịch vụ được chỉ định.

```
1  INT GetAddrInfo(  
2      PCWSTR          pNodeName ,  
3      PCWSTR          pServiceName ,  
4      const ADDRINFOW *pHints ,  
5      PADDRINFOW      *ppResult  
6  )
```

Code 5.12: Cấu trúc của hàm GetAddrInfo

- **pNodeName** tên miền hoặc địa chỉ IP của máy chủ cần truy vấn.

### 5.4.2 WinHttpGetProxyForUrl

**WinHttpGetProxyForUrl** được sử dụng để lấy thông tin proxy cho một URL.

```
1  BOOL WinHttpGetProxyForUrl(  
2      HINTERNET          hSession ,  
3      LPCWSTR            lpCwszUrl ,  
4      WINHTTP_AUTOProxy_OPTIONS *pAutoProxyOptions ,  
5      WINHTTP_PROXY_INFO  *pProxyInfo  
6  )
```

Code 5.13: Cấu trúc của hàm WinHttpGetProxyForUrl

- **lpCwszUrl** chứa URL của trang web mà proxy cần được tìm kiếm.

## 6.1 Phân tích hệ thống

Sơ đồ cấu trúc của công cụ Sandbox bao gồm 4 thành phần chính sau đây: **Sandbox**, **Interceptor**, **Runner** và **Report** (Hình 6.1).

- **Sandbox** là một được thiết kế để kiểm tra dữ liệu nhận được từ Runner. Nhiệm vụ chính của Sandbox là phân tích dữ liệu được gửi từ Runner và trả về kết quả **True** nếu thông tin đó được xác định là có hại. Trong trường hợp không có thông tin độc hại, Sandbox sẽ trả về **False**. Trong cấu trúc này sẽ có hai cấu trúc con là: **ObjectsManager** và **YaraScanner**.
- **Interceptor** có tác vụ là hook vào các hàm API trong tiến trình Powershell. Interceptor thu thập thông tin về các tham số và giá trị trả về của các hàm API này, sau đó gửi thông tin này về cho Runner để tiếp tục xử lý.
- **Runner** được thiết kế để làm trung gian giữa Interceptor và Sandbox. Nhiệm vụ của Runner là nhận dữ liệu từ Interceptor, sau đó đưa dữ liệu vào Sandbox để xử lý. Sau khi nhận kết quả trả về từ Sandbox, Runner gửi kết quả đó cho Interceptor. Ngoài ra, Runner còn chịu trách nhiệm lưu trữ các thông tin về hành vi của chương trình, được gọi là Report.
- **Report** là một thành phần để lưu trữ thông tin về hành vi của mã độc. Report nhận thông tin từ Runner và sau khi kết thúc quá trình, nó sẽ ghi tất cả thông tin này ra file **report.json**.
- **Detector** dùng để phân tích hành vi của mã độc dựa vào kết quả đã được ghi nhận từ Report.

### 6.1.1 Sandbox

Thành phần Sandbox trong hệ thống có nhiệm vụ xử lý dữ liệu được gửi từ Runner. Cụ thể, Sandbox nhận dữ liệu từ Runner dưới hai dạng chính:

- Nhận dữ liệu dưới dạng đường dẫn của file hoặc registry khi tạo hoặc mở.

- Dữ liệu được ghi hoặc đọc ghi trong quá trình thực thi PowerShell.

Sau khi nhận dữ liệu, Sandbox sẽ phân tích và kiểm tra nó để xác định xem liệu dữ liệu này có mang tính độc hại hay không. Quá trình phân tích sẽ trả về kết quả dưới dạng True nếu Sandbox nhận thấy dữ liệu có hại, ngược lại sẽ trả về False.

Cấu trúc cơ bản của lớp Sandbox bao gồm hai đối tượng là quá trình xử lý dữ liệu và quá trình phân tích dữ liệu, đảm bảo rằng dữ liệu được gửi từ Runner được kiểm tra một cách toàn diện và hiệu quả trước khi được tiếp tục xử lý trong hệ thống.

Cấu trúc cơ bản của lớp Sandbox gồm hai đối tượng:

```

1  class Sandbox:
2      def __init__(self) -> None:
3          self.om = ObjectsManager()
4          self.ya = YaraScanner()
5
6      def filter(self, handle: int, object: Object):
7          # checking the path of file or registry
8
9      def scan_memory(self, handle: int, data: bytes):
10         # scan memory of a object when written or read

```

Code 6.1: Cấu trúc của lớp Sandbox

- **filter** là một hàm được dùng để kiểm tra đường dẫn của một file hoặc registry được tạo hoặc mở trong quá trình thực thi.
- **scan\_memory** là một hàm được sử dụng để kiểm tra dữ liệu được ghi vào đối tượng. Hàm sẽ xác định đối tượng được ghi này thông qua Handle bằng cách kiểm tra nó trong ObjectsManager. Nếu có tồn tại, thì bắt đầu kiểm tra nội dung dữ liệu được ghi vào đối tượng này.

#### 6.1.1.1 ObjectsManager

**ObjectsManager** là một cấu trúc được dùng để lưu trữ các Handle của các đối tượng khi chương trình tạo hoặc mở file, registry. Handle là một giá trị dùng để xác định một đối tượng cụ thể được mở trong hệ thống máy tính. Giá trị này được tạo và quản lý bởi hệ điều hành, người dùng chỉ có thể truy cập và thay đổi nội dung của đối tượng thông qua giá trị này. ObjectsManager sẽ quản lý các Handle được trả về cùng với một Object (do công cụ tạo) để đại diện cho đối tượng được mở đó.

```

1  class ObjectsManager:
2      def __init__(self) -> None:
3          self.handle_table: dict[int, Object] = {}

```

Code 6.2: Cấu trúc của lớp ObjectsManager

- **handle\_table** được sử dụng để lưu trữ các đối tượng và ánh xạ chúng với một handle. Key của từ điển là các Handle (kiểu int), và value tương ứng là các đối tượng (Object).

```

1  class Object:
2      def __init__(self) -> None:
3          self.buffer: bytes = None
4      def get_name(self):
5          pass
6      def set_buffer(self, content, offset=-1) -> None:
7          pass
8      def read_buffer() -> bytes:
9          return self.buffer
10 class ObjectFile(Object):
11     def __init__(self, filename: str) -> None:
12         super().__init__()
13         self.filename = filename
14     def get_name(self) -> str:
15         return self.filename
16 class ObjectRegistry(Object):
17     def __init__(self, keyname) -> None:
18         super().__init__()
19         self.keyname = keyname
20     def get_name(self) -> str:
21         return self.keyname

```

Code 6.3: Cấu trúc của lớp Object và các lớp kế thừa

Lớp Object là một lớp cơ bản được sử dụng để biểu diễn các đối tượng file hoặc registry. Đối tượng này được tạo khi có file hoặc registry được tạo hoặc mở trong chương trình. Các lớp kế thừa như **ObjectFile** và **ObjectRegistry** mở rộng lớp này để thêm các thuộc tính và phương thức cụ thể liên quan đến file và registry. Lớp này chứa các thuộc tính và phương thức chung cho việc quản lý và thao tác dữ liệu, cụ thể là thuộc tính **buffer** để lưu trữ dữ liệu của các đối tượng khi ghi vào hoặc đọc từ file hoặc registry. Điều này giúp dễ dàng quản lý và thao tác dữ liệu một cách linh hoạt và hiệu quả.

### 6.1.1.2 YaraScanner

Cấu trúc này được thiết kế để kiểm tra dữ liệu bằng cách sử dụng các quy tắc YARA. Được dùng để quét nội dung của vùng nhớ trong quá trình đọc hay ghi nội dung vào file hoặc registry. Với tính năng đặc biệt này, nó có khả năng nhận diện các mẫu đặc trưng của các loại file và dữ liệu độc hại. Có thể nhận biết được các loại file phổ biến như: EXE, DLL, ZIP, ...

```

1  import yara
2  class YaraScanner:
3      def __init__(self) -> None:
4          filepath = './rules/rules_file.yar'
5          self.rules = yara.compile(filepath=filepath)
6          self.matches = list()
7
8      def scan_memory(self, data: bytes) -> bool:
9          if not isinstance(data, bytes): return False
10         self.matches = self.rules.match(data=data)
11         return True if len(self.matches) != 0 else False

```

```

12
13     def get_matches(self) -> list:
14         return self.matches

```

Code 6.4: Cấu trúc của lớp YaraScanner

- **rules** là một đối tượng của YARA được biên dịch từ một file YARA đã cung cấp. Bộ quy tắc này sẽ được sử dụng để quét dữ liệu và phát hiện các mẫu tương ứng.
- **matches** một danh sách chứa các kết quả của quét, được sắp xếp theo thứ tự mà chúng được tìm thấy trong dữ liệu.
- **scan\_memory** phương thức này nhận đầu vào là dữ liệu dạng bytes cần quét. Nó kiểm tra xem dữ liệu có phải là bytes không. Nếu không phải, phương thức trả về False. Sau đó, nó thực hiện quét dữ liệu bằng cách sử dụng quy tắc YARA và lưu trữ kết quả vào danh sách matches. Phương thức trả về True nếu có ít nhất một kết quả khớp được tìm thấy và False nếu không tìm thấy kết quả nào.
- **get\_matches** phương thức này trả về danh sách matches chứa các kết quả của kiểm tra dữ liệu.

## 6.1.2 Interceptor

Trong hệ thống, vai trò của lớp **Interceptor** không chỉ là giám sát mà còn là điểm trung chuyển quan trọng giữa Process PowerShell và công cụ Sandbox. Nó chịu trách nhiệm ghi nhận và truyền tải thông tin từ các hàm API đến Runner, và sau đó nhận lại kết quả từ Runner để thực hiện các chỉnh sửa cần thiết trước khi trả về kết quả cuối cùng.

Khi một hàm API được gọi, Interceptor sẽ giám sát và lấy các tham số cần thiết, như các tham số truyền vào và giá trị trả về của hàm. Thông tin này sau đó được gửi về cho Runner để thực hiện xử lý.

Runner sẽ xử lý thông tin được nhận và trả về kết quả cho Interceptor. Dựa vào kết quả này, Interceptor có thể thực hiện các chỉnh sửa cần thiết trước khi trả về cho Process PowerShell. Các chỉnh sửa này có thể bao gồm thay đổi tham số truyền vào, thay đổi giá trị trả về, hoặc thậm chí là việc chặn hoặc chuyển hướng luồng thực thi của hàm. Với khả năng can thiệp này, Interceptor có thể đóng vai trò quan trọng trong việc bảo vệ hệ thống khỏi các cuộc tấn công hoặc thực hiện các biện pháp an toàn như giảm thiểu tác động của các hành vi độc hại.

```

1     import frida, sys, os
2     class Interceptor:
3         def __init__(self, pid) -> None:
4             source = self.__build__()
5             self.session = frida.attach(int(pid))
6             self.script = self.session.create_script(source)
7
8         def __build__(self) -> str:
9             script_js = ""

```

```

10         for root, _, files in os.walk('scripts_hook'):
11             for file in files:
12                 hook_script = os.path.join(root, file)
13                 script_js += open(hook_script, 'r').read()
14         return script_js
15
16     def on_detached(self, func) -> None:
17         self.script.on('detached', func)
18
19     def run(self) -> None:
20         self.script.load()
21         sys.stdin.read()
22
23     def send(self, data) -> None:
24         self.script.post(data)
25
26     def recv(self, func) -> None:
27         self.script.on('message', func)

```

Code 6.5: Cấu trúc của lớp Interceptor

- **\_\_build\_\_** ghi tất cả các file script trong thư mục **scripts\_hook** thành một script duy nhất.
- **run** phương thức được dùng để chạy script Frida.
- **send** phương thức được sử dụng để gửi dữ liệu từ Runner cho Interceptor trong quá trình thực thi.
- **recv** phương thức được dùng để nhận dữ liệu từ Process PowerShell thông qua hàm callback **func**.

### 6.1.3 Runner

**Runner** là một đối tượng trung gian của hai cấu trúc Interceptor và Sandbox. Cấu trúc này nhận dữ liệu từ một đối tượng sau đó xử lý trước khi gửi lại cho đối tượng còn lại.

```

1     sandbox = Sandbox()
2     interceptor = Interceptor(PID)
3     report = Report()
4     def file(payload: dict, data = None):
5         .....
6     def registry(payload: dict, data = None):
7         .....
8     def internet(payload: dict):
9         .....
10    def process(payload: dict):
11        .....
12    def on_message(message, data):
13        if message['type'] == 'send':
14            payload = message['payload']

```



```

15         keys = list(payload.keys())
16         if 'File' in keys[0]:
17             file(payload, data)
18         elif 'Reg' in keys[0]:
19             registry(payload, data)
20         elif 'Process' in keys[0]:
21             process(payload)
22         else:
23             internet(payload)

```

Code 6.6: Cấu trúc của Runner

Runner sẽ nhận dữ liệu từ Interceptor thông qua hàm **on\_message**, kiểm tra dữ liệu được gửi từ các hàm nào rồi sau đó đưa qua các hàm tương ứng để tiếp tục xử lý.

- **file** hàm xử lý dữ liệu của các API liên quan tới file.
- **registry** hàm xử lý dữ liệu của các API liên quan tới registry.
- **process** hàm xử lý dữ liệu của các hàm API liên quan tới Process.
- **internet** hàm xử lý dữ liệu của các API liên quan tới mạng.

#### 6.1.4 Report

Dùng để ghi nhận hành vi của mã độc trong quá trình thực thi. Sau khi quá trình thực thi kết thúc thì sẽ ghi nội dung ra file **report.json**. Cấu trúc của **Report** có dạng như sau.

```

1     import json
2     class Report:
3         def __init__(self) -> None:
4             self.record = {}
5             self.id = 1
6         def add_record(self, data) -> None:
7             self.record[self.id] = data
8             self.id += 1
9         # write report variable to json file
10        def dump(self) -> None:
11            f = open('report.json', 'w')
12            f.write(json.dumps(self.report, indent=4))

```

Code 6.7: Cấu trúc của lớp Report

- **add\_record** ghi dữ liệu thu được vào biến record.
- **dump** ghi tất cả dữ liệu đã được lưu ra file report.json.

Cấu trúc của một file report.json để ghi nhận hành vi của chương trình sau khi thực thi có dạng như Hình 6.2.

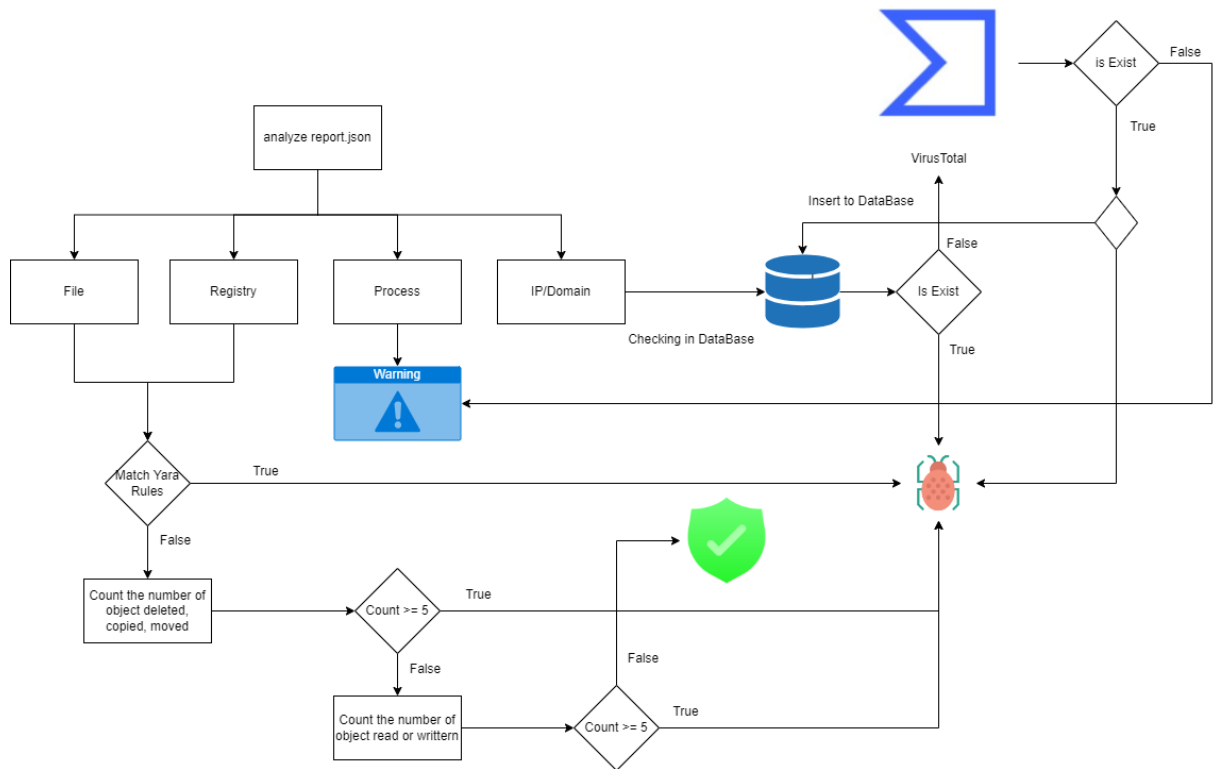
```
"1": {
  "OpenFile": "C:\\Users\\LAP12512-local\\Desktop\\test\\hello.txt",
  "Handle": 4332
},
"2": {
  "CreateFile": "C:\\Users\\LAP12512-local\\Desktop\\test\\hello.txt.encrypted",
  "Handle": 4336
},
"3": {
  "ReadFile": "C:\\Users\\LAP12512-local\\Desktop\\test\\hello.txt",
  "Handle": 4332
},
"4": {
  "ReadFile": "C:\\Users\\LAP12512-local\\Desktop\\test\\hello.txt",
  "Handle": 4332
},
"5": {
  "WriteFile": "C:\\Users\\LAP12512-local\\Desktop\\test\\hello.txt.encrypted",
  "Handle": 4336,
  "Yara Detector": null
},
"6": {
  "DeleteFile": "C:\\Users\\LAP12512-local\\Desktop\\test\\hello.txt"
},
```

Hình 6.2: Cấu trúc của report.json

### 6.1.5 Detector

Cấu trúc này dùng để phân tích hành vi của mã độc thông qua việc phân tích report sau khi chương trình thực thi. Dựa vào kết quả phân tích, cấu trúc này sẽ trả về một dòng chữ được in ra màn hình cho biết hành vi của script PowerShell. Hành vi của mã độc sẽ được phân loại vào các nhóm: Malicious (gây hại), Suspicious (đáng ngờ) và Harmless (vô hại).

- **Malicious** các hành vi rõ ràng gây hại cho hệ thống, chẳng hạn như xóa tệp tin hệ thống.
- **Suspicious** các hành vi đáng ngờ nhưng chưa đủ rõ ràng để kết luận là gây hại. Ví dụ như tạo Process, ...
- **Harmless** các hành vi không gây hại và không có dấu hiệu bất thường. Đây có thể là các hành động thông thường mà không ảnh hưởng đến bảo mật của hệ thống.



Hình 6.3: Sơ đồ hoạt động của quá trình Detector

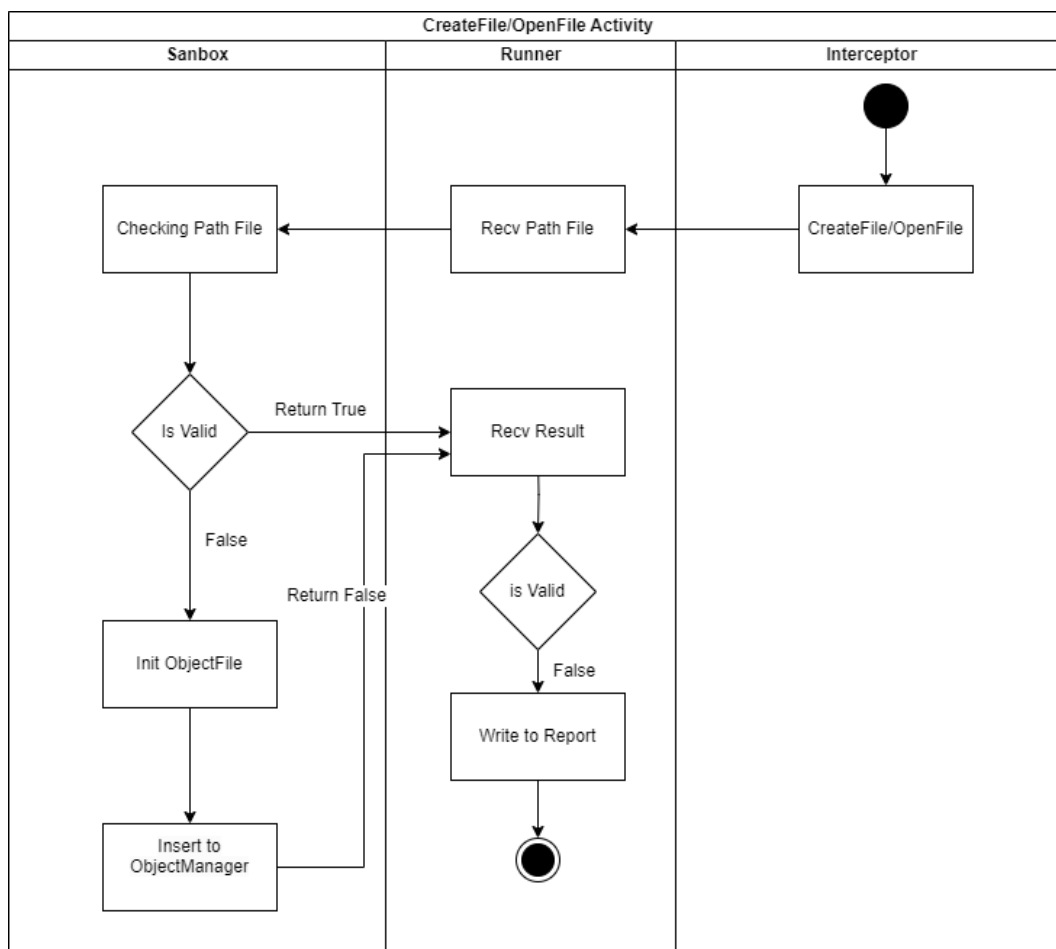
Quá trình phân tích được thực hiện theo các bước sau đây, người đọc có thể tham khảo hình 6.3:

1. Đọc và phân tích dữ liệu lấy từ Report.
2. Nếu có kết nối Internet thì lấy tên miền (Domain) hoặc địa chỉ IP này kiểm tra trong Database, nếu có thì kết luận đó là Malicious, ngược lại nếu trong Database không có thì bắt đầu kiểm tra trên Virustotal. Nếu kiểm tra được Domain hoặc IP đó là Malicious thì kết luận Malicious và cập nhật dữ liệu này vào trong Database. Ngược lại nếu kết quả kiểm tra không có Malicious thì kết luận là Suspicious.
3. Nếu PowerShell có tạo Process thì kết luận đó là Suspicious.
4. Các thông tin còn lại là File và Registry, hệ thống kiểm tra theo tuần tự sau. Nếu trong các thông tin này có hành vi ghi (Write) dữ liệu mà dữ liệu này khớp với Yara Rule thì kết luận đó là Malicious. Nếu không thì tiếp tục thực hiện các bước tiếp theo.
5. Đếm số lượng các file bị xóa (DeleteFile), sao chép (CopyFile) hoặc di chuyển (MoveFile). Nếu có số lượng lớn các file bị các tác động nêu trên thì kết luận đó là Malicious. Nếu không thì thực hiện bước tiếp theo.
6. Đếm số lượng các file và registry được đọc hoặc ghi, nếu nhận thấy rằng một số lượng lớn (cụ thể trong công cụ này là 5) các file hoặc registry được đọc hoặc ghi thì kết luận đó là Malicious. Ngược lại thì kết luận là Harmless.

## 6.2 Quá trình xử lý dữ liệu của các thành phần trong hệ thống

Trong mục này, chúng tôi sẽ giải thích và minh họa quá trình xử lý dữ liệu của các hàm API trong hệ thống. Chúng tôi sẽ tập trung phân tích vào các hàm API liên quan tới file, và registry.

### 6.2.1 Quá trình xử lý CreateFile và OpenFile



Hình 6.4: Sơ đồ hoạt động của quá trình CreateFile và OpenFile

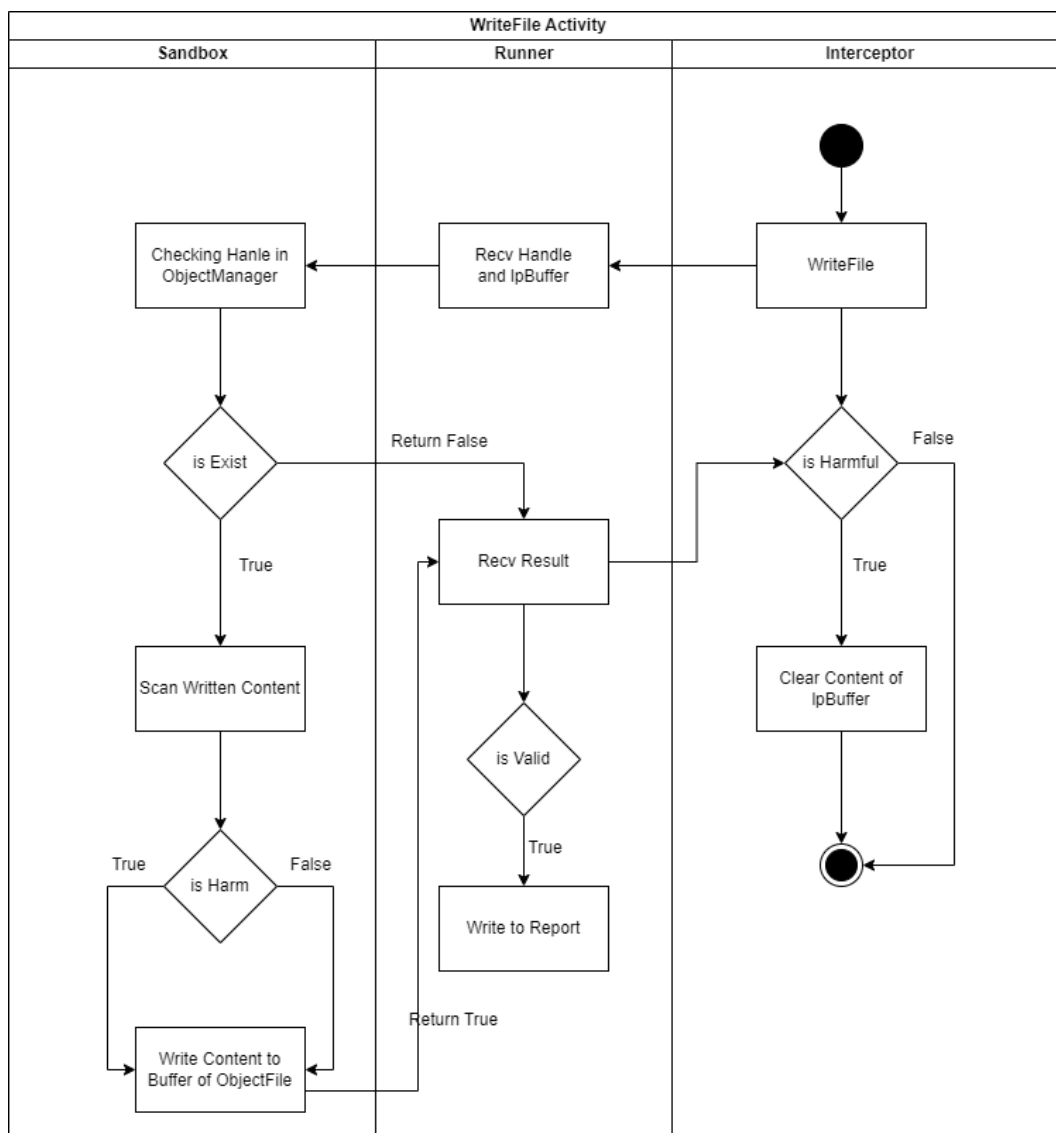
Đối với quá trình tạo file (CreateFile). Interceptor sẽ lấy thông tin của các tham số trong hàm (đã được đề cập ở mục 5.1). Sandbox sẽ kiểm tra đường dẫn của file được tạo thông qua hàm **filter** của Sandbox. Ngoại trừ các file nằm trong các thư mục của hệ thống Windows như: **C:\Windows**, **C:\Program File**, **C:\Program File(x86)**, **C:\Program Data** thì các đường dẫn khác đều là không hợp lệ. Lúc này Sandbox sẽ lưu lại Handle của file và **ObjectFile** được tạo bởi hệ thống vào trong **ObjectsManager**. Đối với trường hợp này thì file vẫn sẽ được tạo, bởi nếu file không được tạo thì trong quá trình thực thi chương trình có thực hiện mở file mà file này không tồn tại trong hệ thống thì sẽ phát sinh lỗi và chương trình sẽ

tự động kết thúc. ObjectFile bao gồm tên của file được tạo và được cấp phát một vùng nhớ để lưu trữ nội dung của file đó khi chương trình có thực hiện ghi file.

Đối với quá trình mở file (OpenFile) cũng tương tự như quá trình tạo file. Nếu file không tồn tại trong hệ thống, thì khi thực thi chương trình sẽ phát sinh lỗi.

Runner sẽ ghi lại quá trình CreateFile hoặc OpenFile vào report khi kết quả nhận được từ Sandbox là False (tức File được tạo hoặc mở không hợp lệ).

### 6.2.2 Quá trình xử lý WriteFile



Hình 6.5: Sơ đồ hoạt động của quá trình WriteFile

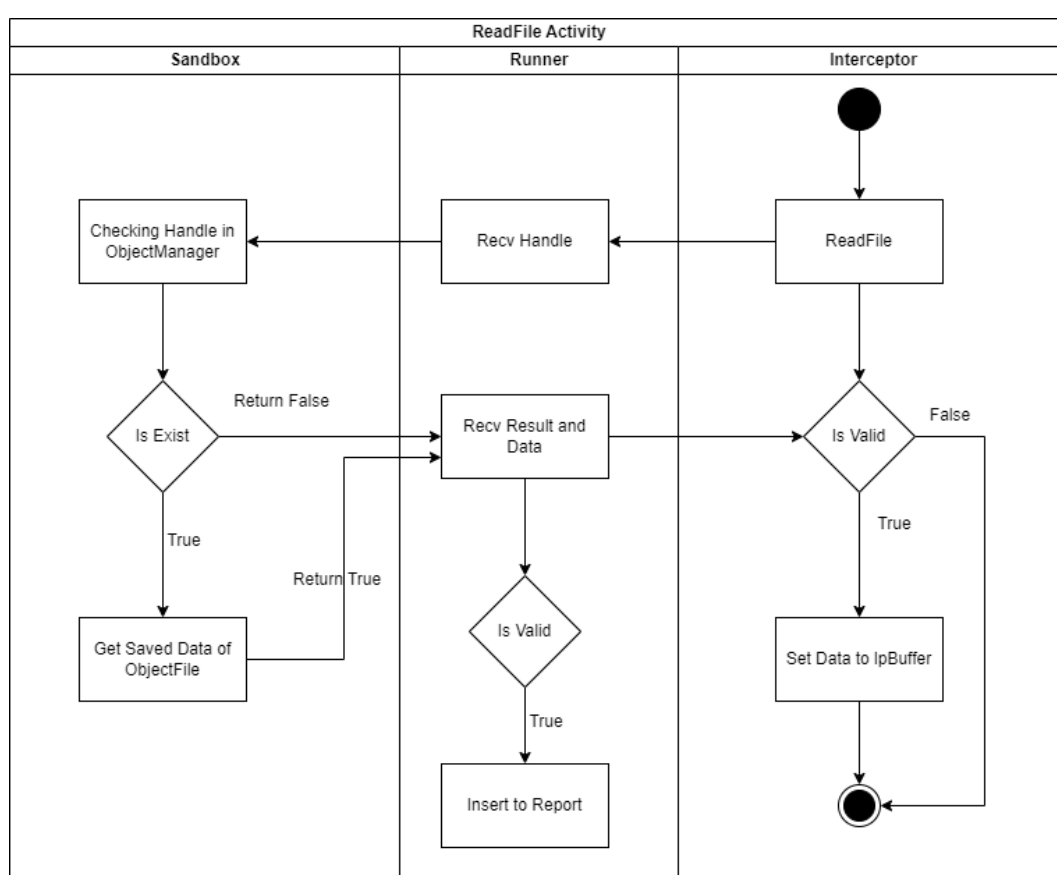
Với quá trình ghi file (WriteFile) sẽ được thực hiện theo hai bước. Đầu tiên, Sandbox sẽ kiểm tra Handle của file được ghi này có tồn tại trong ObjectsManager, nếu có tồn tại thì sẽ chuyển qua bước thứ hai là dùng YaraScanner để quét dữ liệu được ghi thông qua hàm **scan\_memory** của Sandbox. Bước kiểm tra này để xác định nội dung được ghi thuộc về một số loại file thường được mã độc sử dụng hay không, một số loại dữ liệu thường được mã độc Powershell sử dụng như:

DLL, EXE, Base64, ... và nhiều định dạng khác để phát hiện ra bất kỳ dấu hiệu đáng ngờ. Nếu quét được dữ liệu này khớp với Yara rule đã được viết thì lúc này dữ liệu được xem là có hại, Sandbox sẽ trả về giá trị **True**. Ngược lại nếu Handle không tồn tại trong ObjectsManager hoặc quá trình kiểm tra dữ liệu không khớp với Yara thì sẽ trả về giá trị **False**.

Interceptor tiếp tục thực thi dựa vào kết quả trả về từ Sandbox. Nếu là True thì sẽ xóa dữ liệu trong biến **lpBuffer** của hàm WriteFile, ngược lại thì cho phép ghi xuống hệ thống. Phần dữ liệu bị xóa sẽ được lưu vào trong vùng nhớ của ObjectFile đã lưu trước đó.

Runner sẽ ghi lại thông tin quá trình WriteFile vào report nếu kết quả nhận về từ Sandbox là True (tức hành động WriteFile có thể gây hại).

### 6.2.3 Quá trình xử lý ReadFile



Hình 6.6: Sơ đồ hoạt động của quá trình ReadFile

Với quá trình đọc file (ReadFile), Sandbox sẽ kiểm tra Handle của file được đọc này có tồn tại trong ObjectsManager, nếu Handle có tồn tại thì Sandbox sẽ trả về **True** cùng với dữ liệu của Handle cần lấy. Ngược lại thì trả về **False**.

Interceptor nhận dữ liệu được trả về từ Sandbox và tiếp tục xử lý, nếu là True thì sẽ lấy dữ liệu và ghi vào lpBuffer của hàm ReadFile.

Runner sẽ ghi lại thông tin quá trình ReadFile nếu kết quả nhận về từ Sandbox là True.

---

## 6.2.4 Quá trình xử lý DeleteFile, CopyFile và MoveFile

Trong quá trình bảo vệ hệ thống khỏi các hành vi độc hại, công cụ của chúng tôi đặc biệt chú trọng đến các tác vụ liên quan đến quản lý file như xóa file (DeleteFile), sao chép file (CopyFile), và di chuyển file (MoveFile). Các hành vi này thường được mã độc lợi dụng để phá hoại hoặc đánh cắp dữ liệu. Để đảm bảo an toàn tuyệt đối cho hệ thống và dữ liệu của người dùng, công cụ của chúng tôi sẽ ngăn chặn hoàn toàn việc thực hiện các tác vụ này khi được phát hiện. Không chỉ dừng lại ở việc ngăn chặn, công cụ còn ghi nhận lại chi tiết từng hành vi bị ngăn chặn vào report.

## 6.2.5 Các hàm API liên quan tới Registry

Đối với quá trình tạo hoặc mở một key trong registry, Sandbox cũng sẽ kiểm tra đường dẫn của key đó có nằm trong các đường dẫn Persistence<sup>1</sup> thường được mã độc sử dụng hay không. Đường dẫn Persistence cho phép mã độc tự động thực thi khi máy tính được khởi động. Ngoài ra, mã độc còn sử dụng cơ chế tự động lên lịch thực thi của các tác vụ (**Scheduled tasks**) để có thể tự động thực thi mã độc. Nếu key được mở nằm trong các đường dẫn này thì Sandbox sẽ lưu lại Handle của key để theo dõi. Một số khóa registry mà mã độc thường dùng làm Persistence:

- Software\Microsoft\Windows\CurrentVersion\Run
- Software\Microsoft\Windows\CurrentVersion\RunOnce
- Software\Microsoft\Windows\CurrentVersion\RunOnceEx
- Software\Microsoft\Windows NT\CurrentVersion\Winlogon
- Software\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tree
- Software\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tasks

Trong quá trình bảo vệ hệ thống, công cụ của chúng tôi xử lý các tác vụ liên quan đến việc đọc và ghi giá trị vào các khóa registry tương tự như cách nó xử lý việc đọc và ghi file. Các hành vi này thường được mã độc sử dụng để thay đổi cấu hình hệ thống hoặc thiết lập cơ chế Persistence để duy trì các hành vi độc hại. Do đó, công cụ sẽ giám sát chặt chẽ và ngăn chặn mọi thao tác không được phép trên registry, bảo vệ hệ thống khỏi các thay đổi không mong muốn. Vì các quy trình và cơ chế này tương tự với các thao tác đọc và ghi file mà chúng tôi đã trình bày, nên chúng tôi sẽ không đi vào chi tiết về các chức năng trong phần này.

---

<sup>1</sup>Persistence là một kỹ thuật được sử dụng để tự động thực thi mã độc mà không cần sự cho phép của người dùng

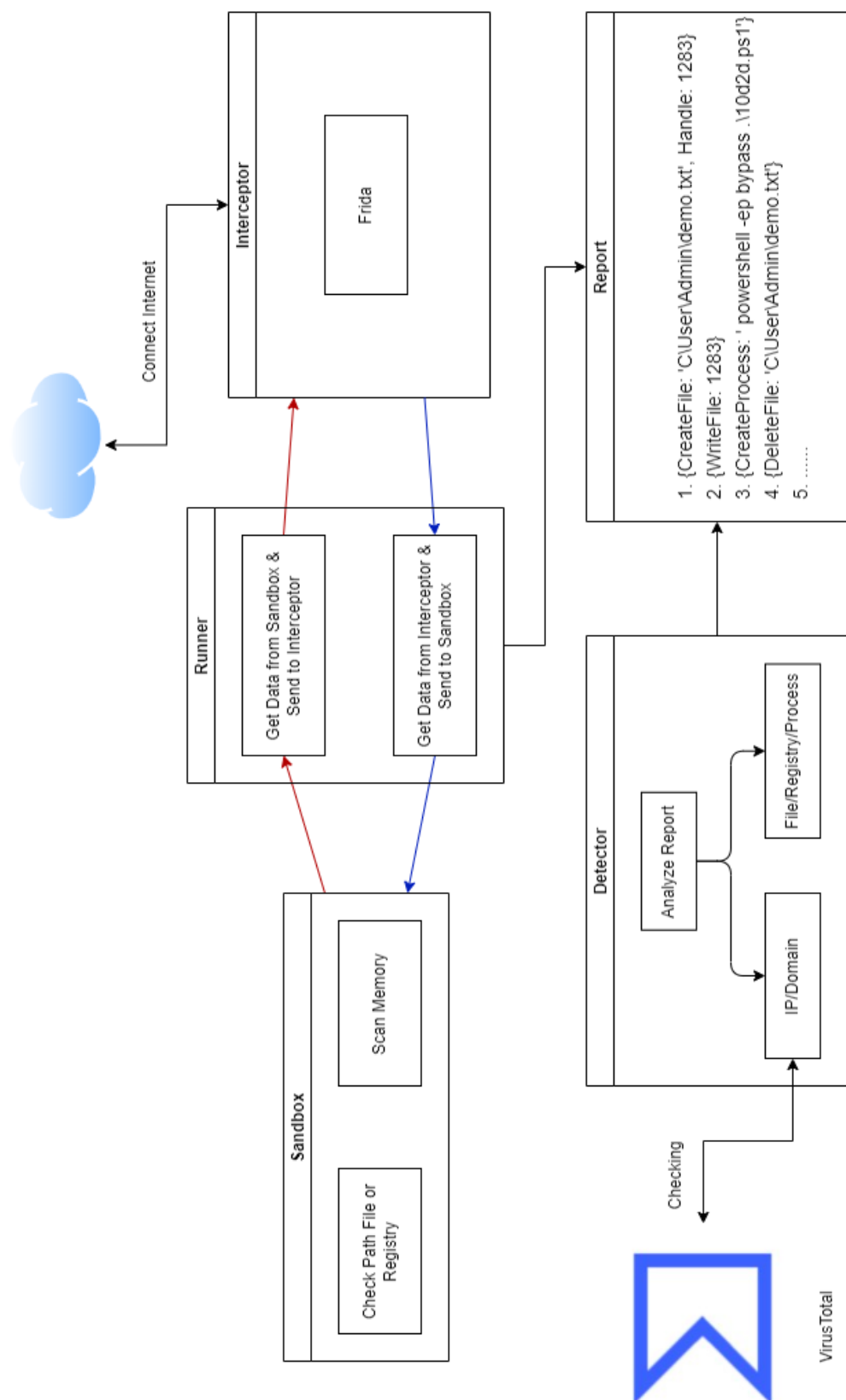
---

### 6.2.6 Các hàm API liên quan tới Process và Internet

Trong quá trình hoạt động của mã độc, việc tạo Process hoặc Thread là một bước quan trọng mà Interceptor của chúng tôi sẽ giám sát chặt chẽ. Khi mã độc cố gắng tạo ra các Process hoặc Thread mới, Interceptor sẽ can thiệp và chặn các hoạt động này để ngăn chặn bất kỳ hành vi độc hại nào có thể gây hại cho hệ thống. Điều này đảm bảo rằng mã độc không thể triển khai hoặc thực thi thêm các thành phần độc hại khác, giới hạn khả năng lây lan và tác động tiêu cực của nó.

Ngược lại, đối với các hoạt động kết nối Internet, công cụ của chúng tôi áp dụng một chiến lược khác. Thay vì chặn hoàn toàn, chúng tôi cho phép mã độc thực hiện các kết nối mạng ra ngoài. Việc này có hai mục tiêu chính. Thứ nhất, cho phép kết nối Internet giúp chúng tôi theo dõi và phân tích các hành vi mạng của mã độc, từ đó hiểu rõ hơn về cách thức hoạt động và mục tiêu của nó. Thứ hai, việc giám sát các kết nối mạng có thể cung cấp thông tin quý giá về các máy chủ (C&C) mà mã độc liên lạc, hỗ trợ chúng tôi phát hiện hành vi của mã độc này.





Hình 6.1: Cấu trúc đề suất của công cụ

## Kiểm thử và Đánh giá công cụ

Trong chương này, chúng tôi sẽ thực hiện kiểm thử và đánh giá quá trình hoạt động của công cụ.

### 7.1 Kiểm thử

Kiểm thử quá trình hoạt động của công cụ trong việc phát hiện từng hành vi cụ thể của một số đoạn PowerShell script do chúng tôi tự viết. Quá trình kiểm thử này bao gồm kết quả do chúng tôi tự phân tích (đã biết trước kết quả), các kết quả còn lại là từ các công cụ đã nghiên cứu và công cụ của chúng tôi đề xuất.

| ID | Đặc trưng của script                                                                    | Tự đánh giá | VirusTotal | AnyRun     | Công cụ đề xuất |
|----|-----------------------------------------------------------------------------------------|-------------|------------|------------|-----------------|
| 1  | Script ghi nội dung đã bị mã hóa base64 vào file và thực thi file đó                    | Malicious   | Malicious  | Harmless   | Malicious       |
| 2  | Chèn một lệnh PowerShell vào Registry, để có thể tự kích hoạt sau mỗi lần khởi động máy | Malicious   | Harmless   | Harmless   | Malicious       |
| 3  | Mở notepad bằng lệnh Start-Process                                                      | Harmless    | Harmless   | Harmless   | Suspicious      |
| 4  | Tải xuống một file từ một địa chỉ IP và ghi vào một file khác                           | Malicious   | Malicious  | Suspicious | Malicious       |
| 5  | Thực thi một lệnh PowerShell để tải xuống một file từ một địa chỉ IP                    | Malicious   | Malicious  | Malicious  | Malicious       |
| 6  | Thực thi một lệnh PowerShell để tải xuống một file và thực thi file đó                  | Malicious   | Harmless   | Suspicious | Malicious       |
| 7  | Thực thi một lệnh để tải xuống một file từ trang Github                                 | Harmless    | Harmless   | Harmless   | Suspicious      |
| 8  | Tải một file PE và thực thi nó                                                          | Malicious   | Malicious  | Suspicious | Suspicious      |
| 9  | Thực thi lệnh PowerShell với option <b>Pypass</b> và <b>Hidden</b>                      | Malicious   | Malicious  | Harmless   | Malicious       |
| 10 | Tải một file từ trang github                                                            | Malicious   | Malicious  | Harmless   | Suspicious      |
| 11 | Thực thi một process                                                                    | Malicious   | Malicious  | Harmless   | Suspicious      |
| 12 | Tạo một khóa trên registry                                                              | Malicious   | Malicious  | Suspicious | Malicious       |
| 13 | Ghi một file PE vào file và thực thi file đó                                            | Malicious   | Malicious  | Suspicious | Malicious       |
| 14 | Thực thi process với chế độ ngầm (Hidden)                                               | Malicious   | Malicious  | Harmless   | Malicious       |
| 15 | Đọc nội dung của một file                                                               | Harmless    | Harmless   | Harmless   | Harmless        |
| 16 | Xóa một giá trị bất kì trong Registry                                                   | Malicious   | Harmless   | Harmless   | Malicious       |

Bảng 7.1: Danh sách các mẫu đã được kiểm tra

| ID | Đặc trưng của script                                                                             | Tự đánh giá | VirusTotal | AnyRun    | Công cụ đề xuất |
|----|--------------------------------------------------------------------------------------------------|-------------|------------|-----------|-----------------|
| 17 | Xóa tất cả file trong thư mục Desktop                                                            | Malicious   | Malicious  | Malicious | Malicious       |
| 18 | Copy nội dung của Folder User ra Folder C:\Encrypted                                             | Malicious   | Malicious  | Harmless  | Malicious       |
| 19 | Ghi nội dung PowerShell đã bị Obfuscation vào file, và thực thi file với chế độ hidden và bypass | Malicious   | Malicious  | Harmless  | Malicious       |
| 20 | Ghi nội dung an toàn vào một file bất kì                                                         | Harmless    | Harmless   | Harmless  | Harmless        |

Bảng 7.2: Danh sách các mẫu đã được kiểm tra (Tiếp theo)

Sau khi tiến hành kiểm tra trên 20 mẫu thử, chúng tôi đã thu được kết quả sau:

|            | Tự đánh giá | VirusTotal | AnyRun | Công cụ đề xuất |
|------------|-------------|------------|--------|-----------------|
| Malicious  | 15          | 13         | 2      | 13              |
| Suspicious | 0           | 0          | 5      | 5               |
| Harmless   | 5           | 7          | 13     | 2               |

Bảng 7.3: Bảng tổng kết

Từ bảng tổng kết 7.3, các công cụ đã nghiên cứu và công cụ đề xuất đều có những khác biệt nhất định so với kết quả tự đánh giá. Tuy nhiên, mỗi công cụ có những phương pháp và tiêu chí riêng để phân loại, do đó sự khác biệt này là dự kiến. Để có cái nhìn toàn diện hơn, chúng tôi đã tính toán độ chính xác của các công cụ bằng phương pháp phân lớp. Bởi vì công cụ này chạy trên máy thật, nên để đảm bảo tính an toàn tuyệt đối cho người dùng, chúng tôi đã gom hành vi đáng ngờ (Suspicious) vào chung với hành vi độc hại (Malicious).

Công thức tính độ chính xác dựa vào công thức sau:

$$Accuracy = \frac{TP + TN}{Total}$$

Trong đó:

- Dương tính thật (True Positive): là những trường hợp phát hiện đúng các mẫu có hại so với thực tế.
- Âm tính thật (True Negative): là những trường hợp phát hiện đúng các mẫu không có hại so với thực tế.

- Số mẫu kiểm thử (Total): là tổng số mẫu cần kiểm tra.

|              |    | VirusTotal |          | AnyRun    |          | Công cụ đề xuất |          |
|--------------|----|------------|----------|-----------|----------|-----------------|----------|
|              |    | Malicious  | Harmless | Malicious | Harmless | Malicious       | Harmless |
| Malicious    | 15 | 12         | 1        | 7         | 8        | 15              | 0        |
| Harmless     | 5  | 3          | 4        | 0         | 5        | 3               | 2        |
| Độ chính xác |    | 80%        |          | 60%       |          | 85%             |          |

Bảng 7.4: Đánh giá độ chính xác của mô hình

Kết quả từ bảng 7.4, cho thấy rằng công cụ của chúng tôi có độ chính xác cao hơn so với các công cụ liên quan. Tuy nhiên kết quả đánh giá này trên tập nhỏ, nên chưa thể hiện chính xác độ hiệu quả của công cụ. Do đó cần phải thực hiện kiểm tra trên nhiều mẫu hơn, với các mẫu thực tế để đưa ra kết quả đánh giá khách quan hơn cho công cụ của mình.

## 7.2 So sánh giữa các công cụ đã được nghiên cứu

Chúng tôi đã thực hiện so sánh một số tiêu chí giữa công cụ đề xuất với các công cụ đã được giới thiệu ở tiểu mục 3.2.1.

| Tiêu chí                    | Anti-Virus           | AnyRun  | Microsoft Copilot | Công cụ đề xuất |
|-----------------------------|----------------------|---------|-------------------|-----------------|
| Phí                         | Chỉ có phiên bản Pro | Đăng ký | Đăng ký           | Mã nguồn mở     |
| Phương pháp phát hiện chính | Signature            | Hành vi | AI và ML          | Hành vi         |
| Tĩnh/Động                   | Tĩnh                 | Động    | Tĩnh              | Động            |
| Cô lập môi trường thực thi  | No                   | Yes     | No                | Yes             |

Bảng 7.5: So sánh tiêu chí giữa công cụ đề xuất và các công cụ đã nghiên cứu

Dựa trên bảng so sánh 7.5, có thể thấy rằng công cụ đề xuất của chúng tôi có một số điểm mạnh sau:

- Mã nguồn mở: khác với các công cụ khác yêu cầu phí hoặc đăng ký, công cụ của chúng tôi là mã nguồn mở. Điều này cho phép người dùng tự do tùy chỉnh và phát triển theo nhu cầu của họ.
- Phương pháp phát hiện chính dựa trên hành vi: công cụ của chúng tôi sử dụng phương pháp phát hiện dựa trên hành vi, giống như AnyRun. Điều này giúp

---

công cụ có khả năng phát hiện các mối đe dọa mới và phức tạp hơn so với phương pháp dựa trên chữ ký.

- Phân tích động: công cụ của chúng tôi thực hiện phân tích động, giúp nó có thể phát hiện các mối đe dọa trong quá trình thực thi, không giống như Anti-Virus và Microsoft Copilot chỉ thực hiện phân tích tĩnh.
- Cô lập môi trường thực thi: công cụ của chúng tôi có khả năng cô lập môi trường thực thi, giống như AnyRun. Điều này giúp bảo vệ người dùng khỏi các mối đe dọa có thể gây hại trong quá trình thực thi.

Những điểm mạnh này giúp công cụ của chúng tôi trở thành một lựa chọn đáng xem xét cho những người đang tìm kiếm một giải pháp bảo mật hiệu quả.

## 7.3 Đánh giá

### 7.3.1 Một số tiêu chí đánh giá

Dưới đây là một số tiêu chí mà chúng tôi mong muốn công cụ có thể đạt được sau khi kết thúc giai đoạn Đồ án tốt nghiệp:

1. Có ứng dụng được kỹ thuật Sandbox để phát hiện được mã độc Ransomware hay không.
2. Khả năng cô lập môi trường của Sandbox.
3. Có khả năng nhận biết một số hành vi của mã độc Ransomware được viết bằng PowerShell.
4. Có khả năng thực thi các script PowerShell.
5. Có khả năng thực thi các đoạn mã PowerShell (không có script).

### 7.3.2 Kết quả đạt được

Sau khi hiện thực công cụ, qua các bước kiểm tra và đánh giá, chúng tôi đã thu được một số kết quả sau:

1. Có khả năng nhận biết một số mã độc Ransomware bằng PowerShell ([Đạt](#)).
2. Có khả năng cô lập môi trường thực thi ([Đạt một phần](#)). Vì công cụ vẫn chưa thể cô lập được quá trình tạo file và registry trên hệ thống.
3. Có khả năng nhận biết một số hành vi của mã độc Ransomware được viết bằng PowerShell ([Đạt](#)).
4. Có khả năng thực thi các script PowerShell ([Đạt](#)).
5. Có khả năng thực thi các đoạn mã PowerShell (không có script) ([Đạt](#)).

---

### 7.3.3 Hiện tượng dương tính giả và âm tính giả

Hiện tượng **Dương tính giả** (False Positive) là việc quá trình công cụ nhận định sai hành vi của chương trình đó là mã độc trong khi thực tế thì không phải.

Cụ thể một số trường hợp xảy ra dương tính giả:

- Khi PowerShell thực thi, công cụ đã chặn mọi hành vi liên quan đến việc tạo một Process. tuy nhiên, trong nhiều trường hợp, những hành vi này lại cần thiết cho hoạt động bình thường của hệ thống và không gây bất kỳ nguy hại nào. Điều này dẫn đến việc kết luận về script PowerShell đó gây hại là chưa được chính xác.
- Việc đánh giá số lượng file bị tác động (cụ thể là 5) vẫn chưa được chính xác. Bởi vì chương trình có thể thực hiện một số tác động tới file hoặc registry nhưng các tác động này không gây hại gì cho hệ thống.

Hiện tượng **Âm tính giả** (False Negative) xảy ra khi công cụ không phát hiện ra hành vi nguy hiểm thực sự trong chương trình.

Một số trường hợp xảy ra âm tính giả:

- Việc đánh giá số lượng file bị tác động (cụ thể là 5) vẫn chưa được chính xác. Bởi vì mã độc có thể tải một chương trình khác (Dropper) vào vùng nhớ và có thể trực tiếp thực thi chương trình thông qua vùng nhớ đó. Nếu vậy thì sẽ có rất ít file hoặc thậm chí không có file nào bị tác động nên dẫn đến việc kết luận sai.
- Một số tên miền và địa chỉ IP có thể chưa được báo cáo trên Virustotal nên cũng dẫn đến việc kết luận sai.

Nguyên nhân xảy ra các hiện tượng này là do hệ thống đánh giá của công cụ chưa được chính xác, chưa bao quát hết được tất cả các trường hợp xảy ra ngoài thực tế và các biến thể mới của mã độc.

Trong chương này, chúng tôi sẽ tóm tắt kết quả đạt được trong quá trình hiện thực luận văn tốt nghiệp.

## 8.1 Kết quả đạt được trong quá trình hiện thực luận văn

Trong giai đoạn thực hiện luận văn, chúng tôi đã tập trung nghiên cứu sâu về các vấn đề liên quan đến mã độc Fileless PowerShell, một trong những mối đe dọa bảo mật ngày càng phổ biến và nguy hiểm. Đồng thời tìm hiểu cách phát hiện và cũng như những hạn chế mà hệ thống bảo mật hiện nay đang gặp phải. Dựa trên những kiến thức và kết quả nghiên cứu này, chúng tôi đã đề xuất một giải pháp phù hợp là sử dụng kỹ thuật Sandbox để cô lập quá trình thực thi của mã độc. Từ đó chúng tôi ứng dụng công cụ này trong việc phát hiện hành vi độc hại của mã độc Ransomware được triển khai dưới dạng Fileless PowerShell. Công cụ này đã đem về một số kết quả khả quan. Tuy nhiên ở thời điểm hiện tại vẫn còn một số hạn chế. Trong tương lai, chúng tôi sẽ tiếp tục cải thiện, phát triển và giải quyết các vấn đề còn tồn đọng.

## 8.2 Hạn chế và hướng phát triển trong tương lai

### 8.2.1 Hạn chế

Sau một quá trình nghiên cứu và phát triển, công cụ đã hoạt động và cho ra một số kết quả khả quan. Tuy nhiên, công cụ còn có một số hạn chế nhất định như sau:

- Công cụ vẫn chưa hoàn toàn cô lập.
- Hệ thống phân tích và đánh giá chưa được chính xác.
- Chưa thể phân tích được hành vi của các Process con. Điều này làm cho quá trình nhận dạng mã độc chưa được chính xác.



- 
- Quá trình kết nối Internet chưa kiểm soát được hoàn toàn, do mã độc có thể thực hiện tải một file khác vào vùng nhớ và thực thi nó. Do đó hệ thống sẽ không nhận biết được.
  - Hệ thống vẫn chưa bao quát được các trường hợp tấn công của mã độc Fileless PowerShell trong thực tế.
  - Giao diện sử dụng của công cụ vẫn còn khá thô sơ và chưa thân thiện.

### 8.2.2 Hướng phát triển trong tương lai

Dựa trên những kết quả hiện tại và nhận diện được các hạn chế còn tồn tại, chúng tôi đưa ra một số kế hoạch để cải thiện công cụ trong tương lai như sau:

- Cải thiện công cụ để cô lập hoàn toàn với môi trường bên ngoài.
- Thiết kế một hệ thống đánh giá hành vi của mã độc chính xác hơn. Tương tự như các hệ thống Sandbox online, hệ thống của chúng tôi sẽ đánh giá các hành vi của mã độc và gán điểm số tương ứng. Qua đó, có thể phân loại các mối đe dọa từ ít nguy hiểm đến rất nguy hiểm, giúp việc phát hiện và phản ứng với mã độc trở nên chính xác và hiệu quả hơn.
- Sử dụng các công cụ sandbox online để hỗ trợ quá trình phân tích. Tận dụng nguồn cơ sở dữ liệu khổng lồ trên các hệ thống này để có thể nhận diện nhanh chóng các file đáng ngờ để có thể cảnh báo sớm đến cho người dùng và giảm thiểu rủi ro.
- Công cụ có thể phân tích được các Process con và Thread, bằng cách sử dụng lại các đoạn script đã có.
- Sử dụng AI để nâng cao khả năng phân tích và nhận diện mã độc. AI sẽ được huấn luyện trên một tập hợp lớn các mẫu mã độc, học cách phát hiện và phân loại các mối đe dọa. Việc này không chỉ giúp cải thiện độ chính xác của công cụ mà còn nâng cao khả năng phát hiện các biến thể mới của mã độc mà chưa có trong cơ sở dữ liệu.

## Tài liệu tham khảo

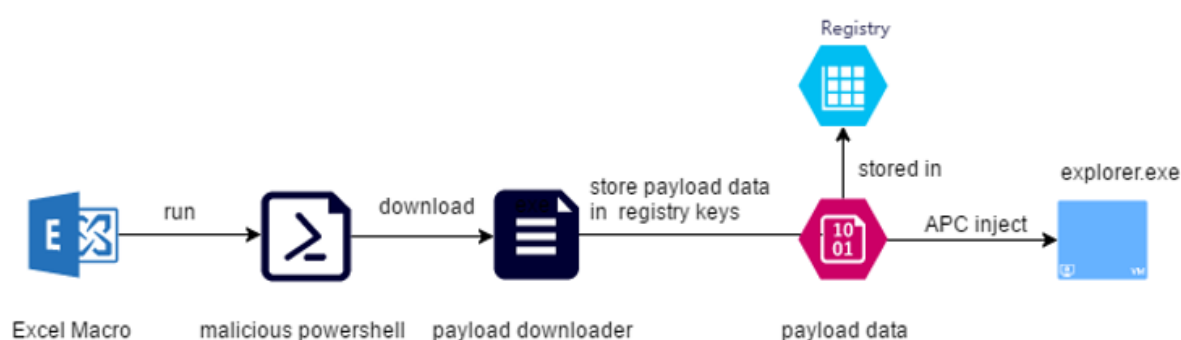
- [1] Saad Ahmed Asad Afreen Moosa Aslam. “Analysis of Fileless Malware and its Evasive Behavior”. In: (2021).
- [2] Sana Aurangzeb et al. “Ransomware: a survey and trends”. In: *Journal of Information Assurance & Security* 6.2 (2017), pp. 48–58.
- [3] BlackFog. *The State of Ransomware in 2023*. URL: <https://www.blackfog.com/the-state-of-ransomware-in-2023/>.
- [4] John Hoopes. *Virtualization for security: including sandboxing, disaster recovery, high availability, forensic analysis, and honeypotting*. Syngress, 2009. Chap. Chapter 3.
- [5] Faiza Babar Khan et al. “Detection of Data Scarce Malware Using One-Shot Learning With Relation Network”. In: *IEEE Access* (2023).
- [6] S Kumar et al. “A survey on various asymmetric algorithms”. In: *International Research 7. Journal of Advanced Engineering and Science* 1.4 (2016), pp. 117–119.
- [7] Michael Maass. “A theory and tools for applying sandboxes effectively”. In: (2016). URL: <https://www.cs.cmu.edu/~mmaass/pdfs/dissertation.pdf>.
- [8] M. Madou, L. Van Put, and K. De Bosschere. “Understanding Obfuscated Code”. In: *14th IEEE International Conference on Program Comprehension (ICPC'06)*. 2006, pp. 268–274. DOI: 10.1109/ICPC.2006.49.
- [9] Giuseppe Mario Malandrone et al. “Powerdecode: a powershell script decoder dedicated to malware analysis”. In: *Proceedings of the Italian Conference on Cybersecurity, ITASEC 2021*. Vol. 2940. CEUR-WS. org. 2021, pp. 219–232.
- [10] McAfee. *Fileless Malware Execution with PowerShell Is Easier than You May Realize*. URL: <https://partners.trellix.com/enterprise/en-us/assets/solution-briefs/sb-fileless-malware-execution.pdf>.
- [11] Microsoft. “Getting started with VBA in Office”. In: (6-08-2022). URL: <https://learn.microsoft.com/en-us/office/vba/library-reference/concepts/getting-started-with-vba-in-office>.

- 
- [12] Microsoft. “PE Format”. In: (24-03-2023). URL: <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>.
  - [13] Nick Montfort. “Obfuscated code”. In: *software studies\ a lexicon* (2008), p. 193.
  - [14] Everett Murdock. *DOS the Easy Way*. EasyWay Downloadable Books, 2008.
  - [15] BN Sanjay et al. “An approach to detect fileless malware and defend its evasive mechanisms”. In: *2018 3rd International Conference on Computational Systems and Information Technology for Sustainable Solutions (CSITSS)*. IEEE. 2018, pp. 234–239.
  - [16] Syed Zainudeen Mohd Shaid and Mohd Aizaini Maarof. “In memory detection of Windows API call hooking technique”. In: *2015 International conference on computer, communications, and control technology (I4CT)*. IEEE. 2015, pp. 294–298.
  - [17] SOC\_Team. [ *Windows Forensic* ] *Malware Persistence*. 30-06-2022. URL: <https://sec.vnpt.vn/2022/06/windows-forensic-malware-persistence/>.
  - [18] Sudhakar and Sushil Kumar. “An emerging threat Fileless malware: a survey and research challenges”. In: *Cybersecurity* 3.1 (2020), p. 1.
  - [19] Cybereason Team. *Fileless Malware 101: Understanding Non-Malware Attacks*. URL: <https://www.cybereason.com/blog/fileless-malware>.
  - [20] C+ Visual and Business Unit. *Microsoft portable executable and common object file format specification*. 1999.
  - [21] Carsten Willems, Thorsten Holz, and Felix Freiling. “Toward Automated Dynamic Malware Analysis Using CWSandbox”. In: *IEEE Security & Privacy* 5.2 (2007), pp. 32–39. DOI: 10.1109/MSP.2007.45.
  - [22] Pavel Yosifovich et al. *Windows Internals: System architecture, processes, threads, memory management, and more, Part 1*. Microsoft Press, 2017.

Dưới đây là một số chiến dịch tấn công có sử dụng Fileless Malware làm giai đoạn trung gian. Hầu hết các cuộc tấn công đều trải qua nhiều giai đoạn khác nhau và Fileless Malware thường sẽ được triển khai ở giai đoạn cuối. Tuy nhiên, trong giai đoạn đầu các file có chứa mã độc sẽ tồn tại trên hệ thống và sau đó mới thực hiện một loạt các giai đoạn phức tạp khác nhau. Fileless Malware không chỉ là PowerShell mà còn là JavaScript, thường sử dụng các file HTA (HTML Application).

## A.1 Mã độc ngân hàng Ursnif năm 2019

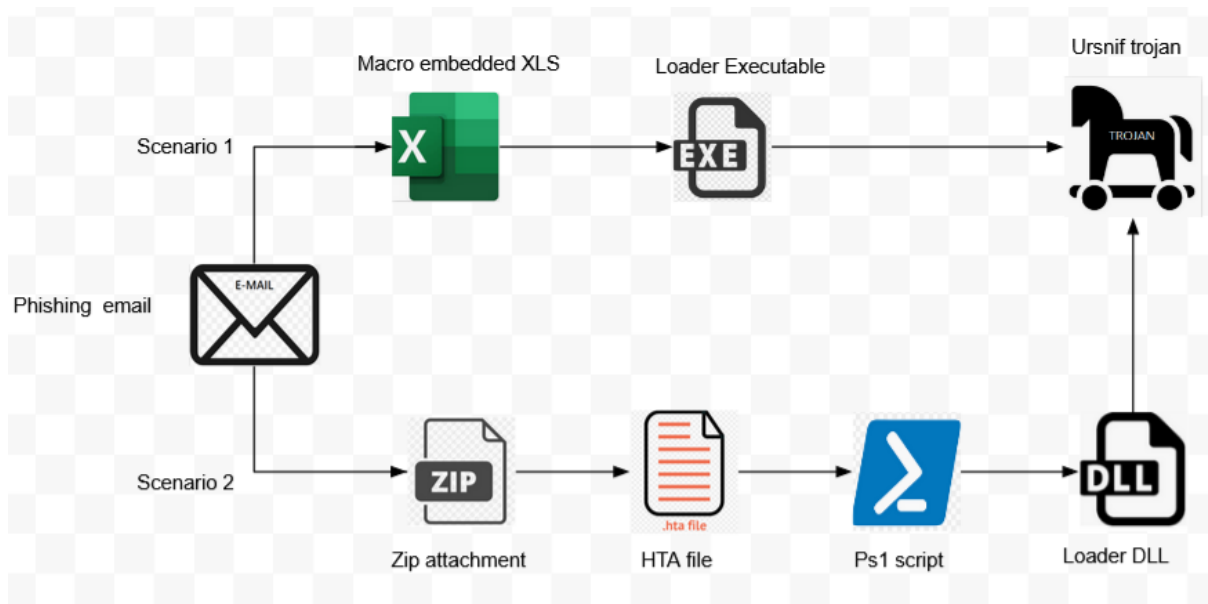
Trong chiến dịch Ursnif<sup>1</sup>, mã độc được phát tán thông qua file Excel có đính kèm các Macro độc hại. Khi Macro này thực thi sẽ triển khai sang dạng mã độc PowerShell để tải một file thực thi (EXE) từ bên ngoài và thực thi nó. Bên cạnh đó, ở giai đoạn PowerShell mã độc còn lưu một phần dữ liệu vào trong Registry.



Hình A.1: Kịch bản tấn công

<sup>1</sup><https://blog.360totalsecurity.com/en/fileless-attack-protection-fully-protect-computer-security/>

## A.2 Mã độc ngân hàng Ursnif năm 2020



Hình A.2: Kịch bản tấn công

Trong chiến dịch này Ursnif<sup>2</sup>, cuộc tấn sẽ được chia làm hai kịch bản sau:

- Kịch bản thứ nhất: file XLS<sup>3</sup> được đính kèm trong mail của nạn nhân, một chương trình mã độc được đính kèm vào trong file XLS, nó chứa một file VBA<sup>4</sup> để thực hiện tải một file thực thi (EXE) từ một tên miền (Domain) đã chuẩn bị sẵn và thực thi file đó.
- Kịch bản thứ hai: một file zip được đính kèm trong mail nạn nhân, trong file zip có đính kèm file HTA. HTA file sẽ triển khai PowerShell script để tải xuống một file DLL và thực thi nó thông qua **rundll32.exe**.

## A.3 Chiến dịch tấn công của tổ chức Gamaredon đầu năm 2022

Trong chiến dịch này, tổ chức Gamaredon<sup>5</sup> đã thực hiện cuộc tấn công dựa trên việc chèn mã độc vào các email liên quan đến Covid-19. Cuộc tấn công cung cấp một tệp tài liệu word dưới dạng tệp đính kèm email. Khi tài liệu được mở, kẻ tấn

<sup>2</sup><https://www.real-sec.com/2022/05/ursnif-malware-banks-on-news-events-for-phishing-attacks>

<sup>3</sup>XLS là một tập tin chứa dữ liệu dạng bảng tính của Microsoft Excel

<sup>4</sup>VBA là ngôn ngữ lập trình của Microsoft Office cho phép thực thi mã bên trong tài liệu để phản hồi một sự kiện cụ thể

<sup>5</sup><https://blog.360totalsecurity.com/en/fileless-attack-protection-fully-protect-computer-security>

công sẽ tải xuống mẫu tài liệu chứa mã Macro độc hại từ Internet. Macro sẽ tiếp tục yêu cầu tải một file VBS độc hại khác và sau đó gửi yêu cầu kết nối tới máy chủ (C&C).



Hình A.3: Kịch bản tấn công

## A.4 Chiến dịch tấn công của tổ chức SideWinder năm 2019

Chiến dịch tấn công của tổ chức SideWinder<sup>6</sup> đã sử dụng kỹ thuật tấn công vùng nhớ bằng ngôn ngữ JavaScript để thực thi các hành vi gây hại. Trước tiên, kẻ tấn công sử dụng lỗ hổng thực thi mã từ xa của Office (cve-2017-11882) để giải mã và thực thi tập lệnh JavaScript, sau đó thực hiện tải một file khác chèn vào vùng nhớ của tiến trình khác và thực thi vùng nhớ đó.



Hình A.4: Kịch bản tấn công

## A.5 Chiến dịch tấn công của tổ chức Bladabindi năm 2021

Trong chiến dịch tấn công của tổ chức Bladabindi<sup>7</sup>, người dùng tải phần mềm miễn phí từ nguồn có hại, sau đó phần mềm yêu cầu cập nhật. Trong quá trình cập nhật sẽ đính kèm một file VBS độc hại. File VBS này sẽ thực thi một chương trình PowerShell để truy cập các tệp độc hại được lưu trữ trên Microsoft Onedrive

<sup>6</sup><https://blog.360totalsecurity.com/en/fileless-attack-protection-fully-protect-computer-security/>

<sup>7</sup><https://blog.360totalsecurity.com/en/fileless-attack-protection-fully-protect-computer-security/>

và sao chép chính nó vào thư mục khởi động (StartUp). Bên cạnh đó PowerShell sẽ tải một file khác từ bên ngoài vào vùng nhớ của chính nó và thực thi.



Hình A.5: Kịch bản tấn công